

(MSCS)

(SISTEMA DE ASIGNACIÓN DE HORARIOS DE PROFESORES)

Software Design Document

Name (s):

Cuapantecatl Ruiz Raul Irving

Lab Section: Laboratorio computo 3 piso 1

Workstation: Diseño de Software

Date: (25/02/2025)

MSCS



TABLA DE CONTENIDOS

Historial de Revisiones

Fecha	Versión	Autor	Razón del rechazo	Modificación realizada
12/02/25	0.1	Cuapantecatl Ruiz Raul Irving		

Historial de Cambio

Versión	Fecha y Horas invertidas	Revisado por	Roll
0.1	12/02/25 - 01:30 Hrs		

Historial de elaboración del documento

Versión	Fecha de Revisión	Realizado por	Actualización se agrega
0.1	12/02/25	Cuapantecatl Ruiz Raul Irving	
0.1	25/02/25	Cuapantecatl Ruiz Raul Irving	introducción y casos de uso
0.1	25/02/25	Cuapantecatl Ruiz Raul Irving	se realiza diagrama de contexto
0.1	05/03/25	Cuapantecatl Ruiz Raul Irving	Se anexa diagramas en cada caso de uso
0.1	06/03/25	Cuapantecatl Ruiz Raul Irving	Se anexa diagramas en cada caso de uso faltantes
0.1	13/03/25	Cuapantecatl Ruiz Raul Irving	Inclusión de diagrama de paquetes
0.1	13/03/25	Cuapantecatl Ruiz Raul Irving	Inclusión de diagrama de componentes

0.1	14/03/25	Cuapantecatl Ruiz Raul Irving	Modificación al diagrama de paquetes
0.1	11/04/25	Cuapantecatl Ruiz Raul Irving	Se realiza Patterns (5.7) Reuse of patterns and available Framework template UML composite structure diagram
0.1	29/04/25	Cuapantecatl Ruiz Raul Irving	Se realiza Interfaz (5.8) Lenguajes de definición de interfaz (IDL), Diagrama de componentes UML
0.1	02/05/25	Cuapantecatl Ruiz Raul Irving	Se realiza Estructura (5.9) Constituyentes internos y organización de los sujetos de diseño, componentes y clases se añade Diagrama de estructura UML, diagrama de clases Interacción (5.10) se añade Diagrama de secuencia UML, diagrama de comunicación UML Comunicación de objetos, mensajería
0.1	09/05/2025	Cuapantecatl Ruiz Raul Irving	Se realiza Dinámica de estados (5.11) Diagrama de máquina de estados UML, diagrama de estados (de Harel), tabla de transición de estados (matriz), autómatas, red de Petri. Transformación dinámica de estados. se anexan diagramas login administrador y profesor, así como agregar profesor. con una breve explicación.
0.1	09/05/2025	Cuapantecatl Ruiz Raul Irving	Se realizan correcciones de diagramas secuenciales. Se incorpora 5.12 Algoritmo Lógica procedimental Tabla de decisiones, diagrama de Warnier, JSP, PDL de los algoritmos para el administrador del sistema de asignación de horarios. SE AÑADE ÍNDICE DEL DOCUMENTO. Así como respectivos diagramas de pseudocódigo..

ÍNDICE:

1. INTRODUCCIÓN	Pág.6
2. Contexto	Pág.6
2.1. Descripción General del Sistema de Asignación de Horarios y	
Materias para Profesores	Pág 6
2.2 Diagrama de Contexto	Pág 8
2.3 Servicios (Casos de Uso)	Pág 8
3. Composición:	Pág.15
3.1 Vista Lógica	Pág.15
3.2 Vista Física	Pág.19
4. Logical del 5.4 IEEE 1016	Pág.21
4.1 Relación con Logical (5.4)	Pág.22
5. Dependency (5.5)	Pág.26
5.1 Diagrama de Paquetes (UML Package Diagram)	Pág.26
5.2 Diagrama de Componentes (UML Component Diagram) ...	
.....	Pág.26
5.3 Relación con Dependency (5.5)	Pág.28
6. Patterns (5.7)	Pág.29
6.1 Patrón Strategy (Estrategia)	Pág.29
6.2 Patrón Observer (Observador)	Pág.31
6.3 Modelo-Vista-Controlador (MVC)	Pág.32
7. Interface (5.8)	Pág.35
7.1 Lenguaje de Definición de Interfaces (IDL)	Pág.35
8. Structure (5.9): Componentes y Organización Interna del Sistema de	
Asignación de Horarios	Pág.38
8.1 Sujetos de diseño	Pág.39
8.2 Componentes del sistema	Pág.39
8.3 Asociación de componentes principales	Pág.40
8.4 Diagrama de Estructura (Clases UML)	Pág.41
8.5 División Arquitectónica	Pág.41
8.6 Clases Funcionales Clave	Pág.42
9. Interacción (5.10)	Pág.44
9.1 DIAGRAMA DE SECUENCIA 1 DEL CASO DE USO 01	
AGREGAR PROFESOR	Pág.44

9.2 DIAGRAMA DE SECUENCIA 2 DEL CASO DE USO 01 BUSCAR PROFESOR	Pág.45
9.3 DIAGRAMA DE SECUENCIA 3 DEL CASO DE USO 01 EDITAR PROFESOR	Pág.48
9.4 DIAGRAMA DE SECUENCIA 4 DEL CASO DE USO 01 BORRAR PROFESOR	Pág.51
10. Dinámica de estados (5.11)	Pág.53
10.1 DIAGRAMA (1) ESTADOS LOGIN PROFESOR.....	Pág.54
10.2 DIAGRAMA (2) ESTADOS LOGIN ADMINISTRADOR.	Pág.55
10.3 DIAGRAMA (3) ESTADOS REGISTRAR PROFESOR..	Pág.56
10.4 DIAGRAMA (4) ESTADOS ASIGNAR HORAS SEMANALES A PROFESOR	Pág.57
10.5 DIAGRAMA (5) ESTADOS ASIGNAR MATERIAS A PROFESOR.....	Pág.58
11. Lógica procedimental algorítmica (5.12)	Pág.59
11.1 Algoritmo 1: Proceso SistemaAsignacionHorarios() (Iniciar sesión)	Pág.60
11.2 Algoritmo 2: Proceso SistemaAsignacionHorarios() (Menú Principal Para Administrador)	Pág.60
11.3 Algoritmo 3: SubProceso AdministrarProfesores() ..	Pág.61
11.4 Algoritmo 4: SubProceso AsignarHorasSemanales().	Pág.62
11.5 Algoritmo 5: SubProceso AsignarMaterias()	Pág.62
11.6 Algoritmo 6: SubProceso AsignarHorarios()	Pág.63
11.7 Algoritmo 7: SubProceso AsignarGrupos()	Pág.64
11.8 Algoritmo 8: SubProceso VisualizarHorarios()	Pág.64
11.9 Algoritmo 9: SubProceso VisualizarMaterias()	Pág.65
11.10 Algoritmo 10: SubProceso VisualizarGrupos()	Pág.65
11.11 Algoritmo 11: Gale_Shapley_AsignacionMaterias()..	Pág.66
12.Conclusión	Pág.67
13. ANEXOS	Pág.67

1. INTRODUCCIÓN

En esta sección, se presentan los puntos de vista de diseño aplicables al **Sistema de Asignación de Horarios y Materias para Profesores**. Estos puntos de vista proporcionan una representación estructurada del sistema desde diferentes perspectivas, permitiendo una mejor comprensión de sus componentes, relaciones y restricciones.

Cada punto de vista está diseñado para abordar preocupaciones específicas del diseño, utilizando lenguajes y notaciones apropiadas para su representación. La aplicación de estos enfoques facilitará la documentación, análisis y validación del sistema en las distintas etapas de su desarrollo.

2. Contexto:

El **Sistema de Asignación de Horarios y Materias para Profesores** es una solución informática orientada a mejorar la gestión académica dentro de instituciones educativas mediante la automatización de tareas clave. Su propósito principal es facilitar y optimizar el proceso de asignación de carga docente, horarios de clases y distribución de grupos estudiantiles, asegurando una administración eficiente, equitativa y sin conflictos.

2.1. Descripción General del Sistema de Asignación de Horarios y Materias para Profesores

La plataforma está diseñada para operar en un entorno controlado donde interactúan dos actores fundamentales: el *Administrador* y el *Profesor*. El Administrador tiene la responsabilidad de configurar y gestionar todos los elementos operativos del sistema, como el registro de profesores, la asignación de materias, la planificación de horarios y la organización de grupos estudiantiles. Por su parte, el profesor puede acceder a la plataforma para consultar su información académica asignada, incluyendo horarios, materias y grupos a cargo.

Desde el punto de vista técnico y funcional, el sistema responde a la necesidad de evitar errores humanos comunes en la planificación académica, como la sobreasignación de horarios, el uso ineficiente de aulas o la carga desequilibrada de

trabajo entre docentes. Para abordar estos retos, el sistema emplea enfoques algorítmicos especializados. Entre ellos destaca el algoritmo de **Gale-Shapley**, utilizado para garantizar emparejamientos estables entre profesores y materias, tomando en cuenta la compatibilidad entre la especialización del docente y las características de las asignaturas. Asimismo, se aplica **programación lógica con restricciones** para validar y resolver conflictos de horarios y disponibilidad de espacios físicos, como aulas o laboratorios, durante el proceso de asignación.

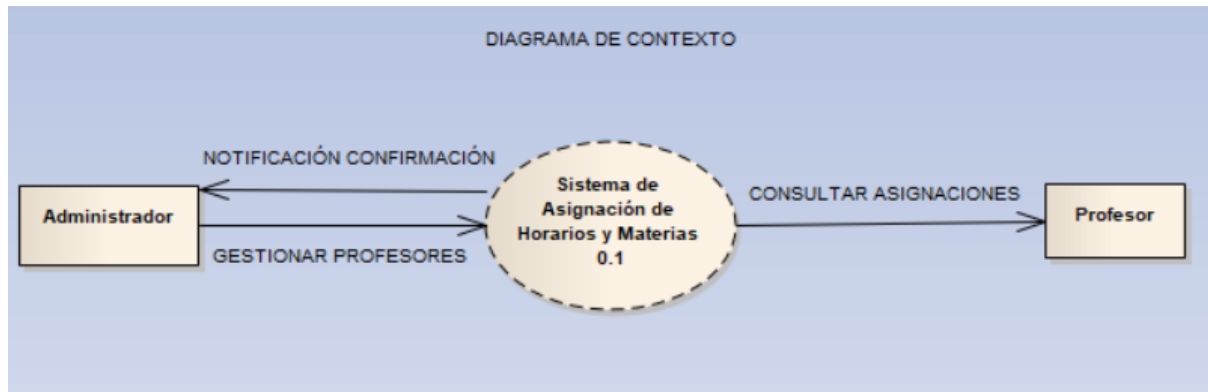
Uno de los pilares del diseño del sistema es la implementación de un modelo de control de acceso basado en roles. Esto significa que cada usuario, según su perfil (administrador o profesor), tendrá acceso únicamente a las funcionalidades que le corresponden, garantizando la seguridad, integridad y confidencialidad de los datos académicos.

Además, se contempla la generación automática de reportes administrativos y académicos, lo que permite al personal directivo contar con información organizada y confiable para la toma de decisiones. Estos reportes son generados por una clase especializada, **GeneradorReportes**, que se apoya en estructuras de datos provenientes de los módulos de eventos y visitas académicas.

El sistema ha sido concebido desde su origen como una herramienta escalable, con un diseño modular que facilita futuras actualizaciones o expansiones. De esta forma, podrá adaptarse a posibles incrementos en la matrícula, el número de docentes o la diversificación del catálogo de materias, sin comprometer su rendimiento ni su estabilidad operativa.

En conjunto, este sistema no solo representa una mejora significativa en los procesos administrativos escolares, sino que también contribuye a una gestión académica más transparente, organizada y centrada en el aprovechamiento eficiente de los recursos disponibles.

2.2 Diagrama de Contexto



- Administrador: Interactúa con el sistema para gestionar la información relacionada con los profesores, asignaciones y horarios.
- Profesor: Utiliza el sistema para consultar su información personal y asignaciones.
- Sistema de Asignación de Horarios y Materias: Proporciona los servicios necesarios para la gestión y consulta de información.

2.3 Servicios (Casos de Uso)

A continuación, se describen los servicios ofrecidos por el sistema, correspondientes a los casos de uso definidos:

Sistema de Asignación de Horarios y Materias para Profesores

1. Actores del Sistema

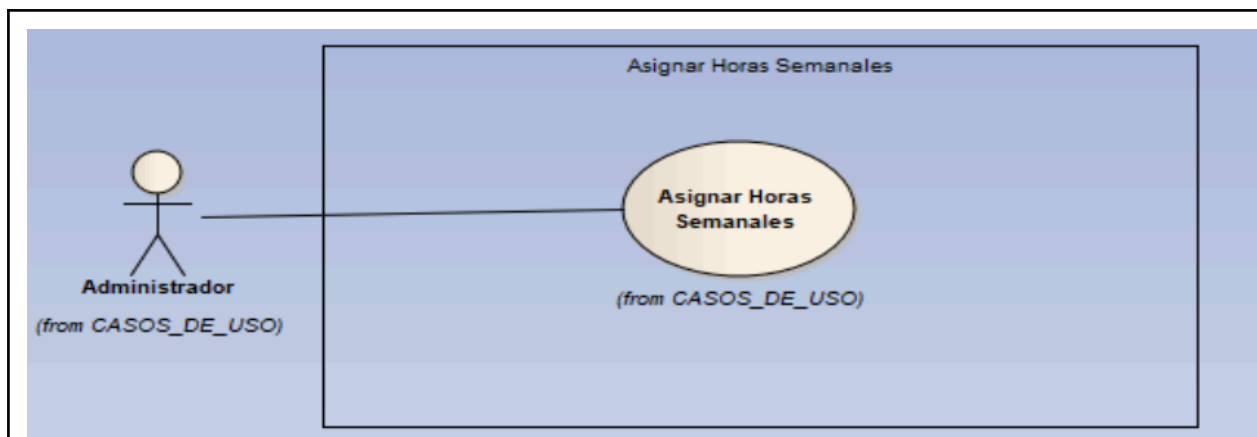
Actores Principales:

- Administrador: Responsable de la gestión de profesores, asignaciones de materias, horarios y grupos de estudiantes.
- Profesor: Usuario que consulta su horario, materias y grupos asignados.

2. Casos de Uso

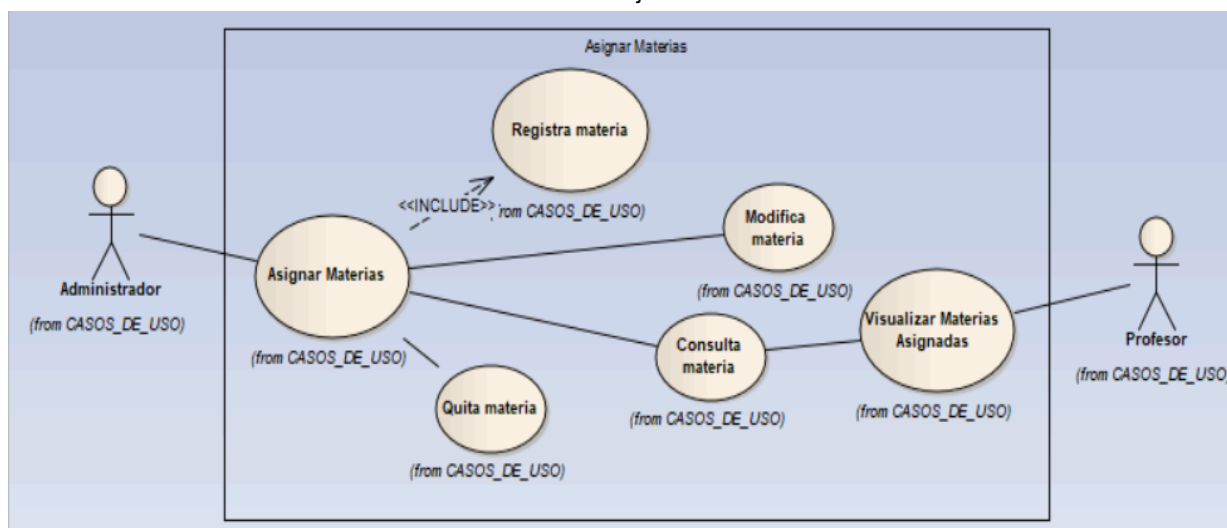
ID	Caso de uso	Descripción	Actor	Flujo Principal
CU01	Administrar Profesores	Permite al administrador agregar, modificar o eliminar profesores.	Administrador	Evento de activación: El administrador accede al módulo de gestión de profesores para agregar, modificar o eliminar profesores

				precondición: El administrador debe estar autenticado en el sistema y tener permisos para gestionar profesores. post-condición: La base de datos se actualiza con los nuevos datos modificados o agregados (Nuevo profesor agregado, datos modificados o eliminar profesor) y muestra un mensaje de confirmación.
El administrador accede a la gestión de profesores. Selecciona la opción en el menú que contiene agregar, buscar, modificar o eliminar. Ingresa los datos del profesor (nombre, correo, especificación de área, número de empleado.). El sistema valida y almacena la información. Se muestra un mensaje de confirmación.				
<pre> graph LR Admin[Administrador (from CASOS_DE_USO)] --- AdminProf[Administrar Profesores (from CASOS_DE_USO)] AdminProf -.-> <<INCLUDE>> (from CASOS_DE_USO) Registrar[Registrar profesor] AdminProf --- Modificar[Modificar profesor (from CASOS_DE_USO)] AdminProf --- Quitar[Quitar profesor (from CASOS_DE_USO)] AdminProf --- Consultar[Consultar profesor (from CASOS_DE_USO)] </pre> The diagram illustrates the 'ADMINISTRAR PROFESORES' use case. An actor named 'Administrador' (from CASOS_DE_USO) is connected to the 'Administrar Profesores' use case (from CASOS_DE_USO). This central use case has four associated use cases: 'Registrar profesor' (from CASOS_DE_USO) connected via an include relationship (<<INCLUDE>>), 'Modificar profesor' (from CASOS_DE_USO), 'Quitar profesor' (from CASOS_DE_USO), and 'Consultar profesor' (from CASOS_DE_USO).				
CU02	Asignar Horas Semanales	Permite asignar una cantidad de horas semanales a los profesores según su especialización.	Administrador	Evento de activación: El administrador accede al módulo de asignación de horas, para asignar la cantidad de horas correspondientes a cada profesor. precondición: El administrador debe estar autenticado en el sistema y tener permisos para gestionar profesores, además de que el profesor debe estar agregado ya en el sistema. postcondición: La base de datos se tiene que actualizar con la cantidad de horas asignadas a cada profesor y muestra un mensaje de confirmación
El administrador accede a la asignación de horas. Selecciona un profesor y establece el número de horas. El sistema valida que no supere el límite permitido. Se guarda la asignación y se muestra un mensaje de confirmación.				



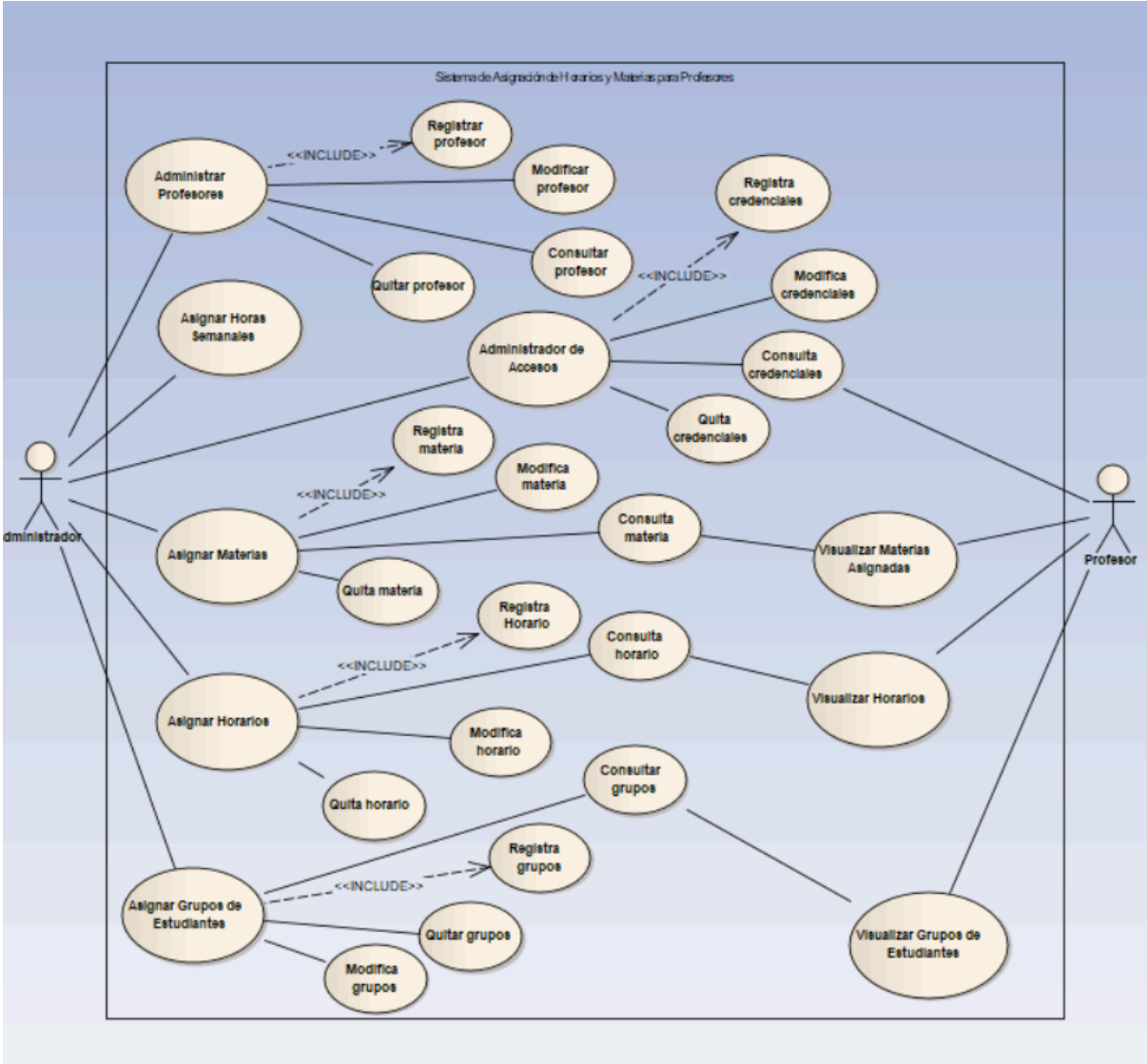
CU03	Asignar Materias	Permite asignar materias a los profesores según su especialización.	Administrador	Evento de activación: El administrador accede al módulo de asignación de materias para asignar una materia a un profesor. precondición: El administrador debe de estar autenticado en el sistema y tener permisos para la gestión de materias, además de que el profesor tiene que estar registrado y contar con la especialización adecuada para la materia. postcondición: La asignación de la materia se guarda en la base de datos y muestra un mensaje de confirmación.
------	------------------	---	---------------	---

El administrador accede a la asignación de materias.
 Selecciona un profesor y la materia correspondiente.
 El sistema valida y guarda la asignación.
 Se muestra un mensaje de confirmación.



CU04	Asignar Horarios	Permite establecer los horarios de cada materia para los profesores.	Administrador	Evento de activación: El administrador accede al módulo de gestión de horarios para establecer los horarios de las materias asignadas a los profesores. precondición: El administrador debe estar autenticado en el sistema y tener permisos para gestionar horarios. El
------	------------------	--	---------------	--

DIAGRAMA DE CASOS DE USOS



Versión	Fecha	Realizado por	Tiempo requerido
0.1	27/02/25	Cuapantecatl Ruiz Raul Irving	3:00 hrs

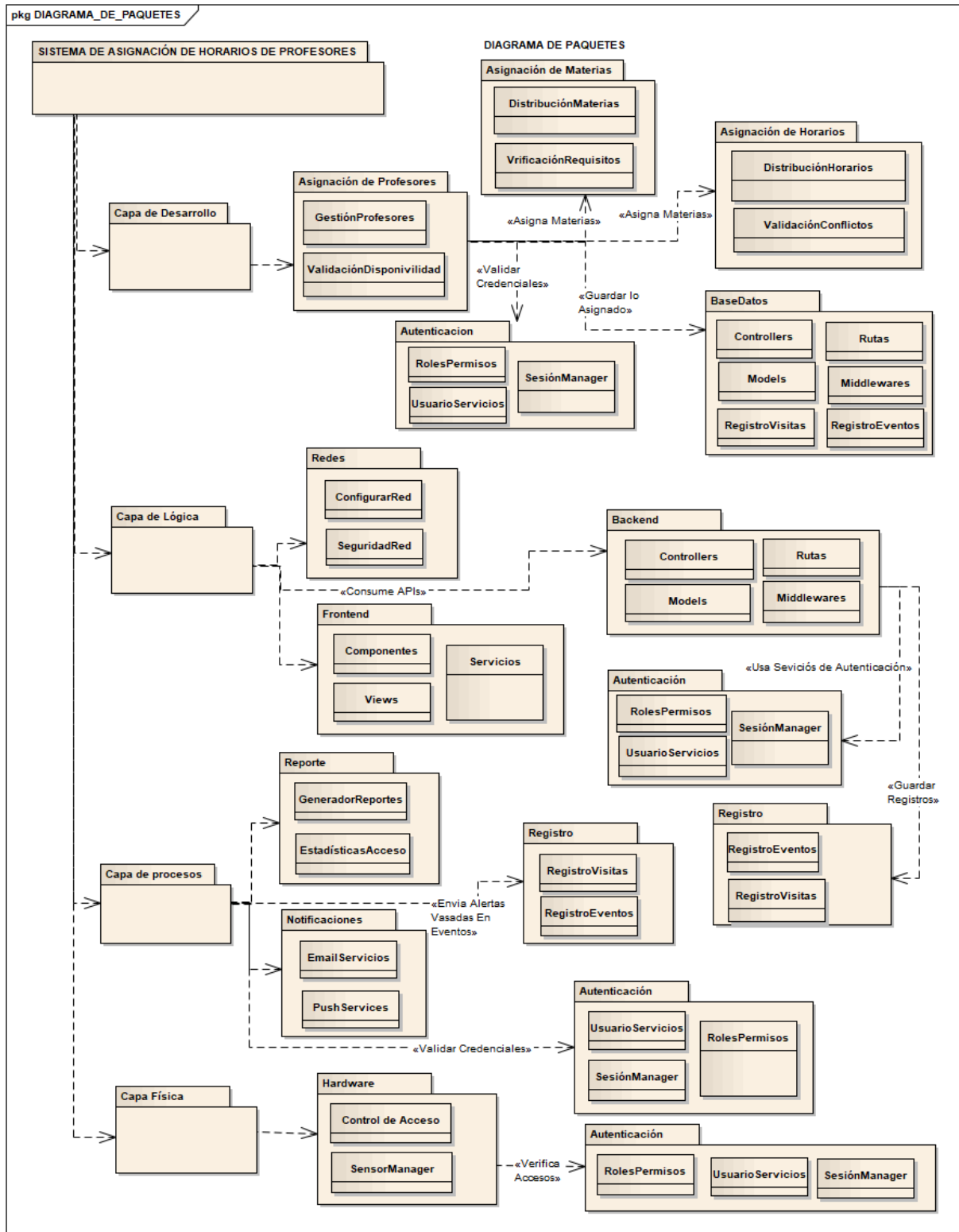
3. Composición:

La composición del sistema se puede refinar en dos puntos de vista principales: lógico y físico.

3.1 Vista Lógica

La vista lógica se centra en la funcionalidad y la estructura del sistema desde una perspectiva de diseño.

- Diagrama de Paquetes UML:
 - Este diagrama mostrará la organización lógica del sistema en paquetes. Se identificarán paquetes como:
- | | |
|--|---|
| <ul style="list-style-type: none">● Capa de Desarrollo<ul style="list-style-type: none">○ Clase Distribución de Materias○ Clase Verificación de Requisitos● Asignación de Horarios<ul style="list-style-type: none">○ Clase Definición de Horarios○ Clase Validación de Conflictos● Asignación de Profesores<ul style="list-style-type: none">○ Clase Gestión de Profesores○ Clase Validación de Disponibilidad● Capa de Lógica● Base de Datos<ul style="list-style-type: none">○ Clase Controllers○ Clase Modelos○ Clase Rutas○ Clase Middlewares○ Clase Registro de Visitas○ Clase Registro de Eventos● Frontend<ul style="list-style-type: none">○ Clase Componentes | <ul style="list-style-type: none">○ Clase Views○ Clase Servicios● Redes<ul style="list-style-type: none">○ Clase Configurar Red○ Clase Seguridad de Red● Capa de Procesos● Autenticación<ul style="list-style-type: none">○ Clase Usuario Servicios○ Clase Sesión Manager○ Clase Roles y Permisos● Registros<ul style="list-style-type: none">○ Clase Registro de Visitas○ Clase Registro de Eventos● Notificaciones<ul style="list-style-type: none">○ Clase Email Servicios○ Clase Push Services● Reporte<ul style="list-style-type: none">○ Clase Generador de Reportes○ Clase Estadísticas de Acceso● Capa Física● Hardware<ul style="list-style-type: none">○ Clase Control de Acceso○ Clase Sensormanager |
|--|---|



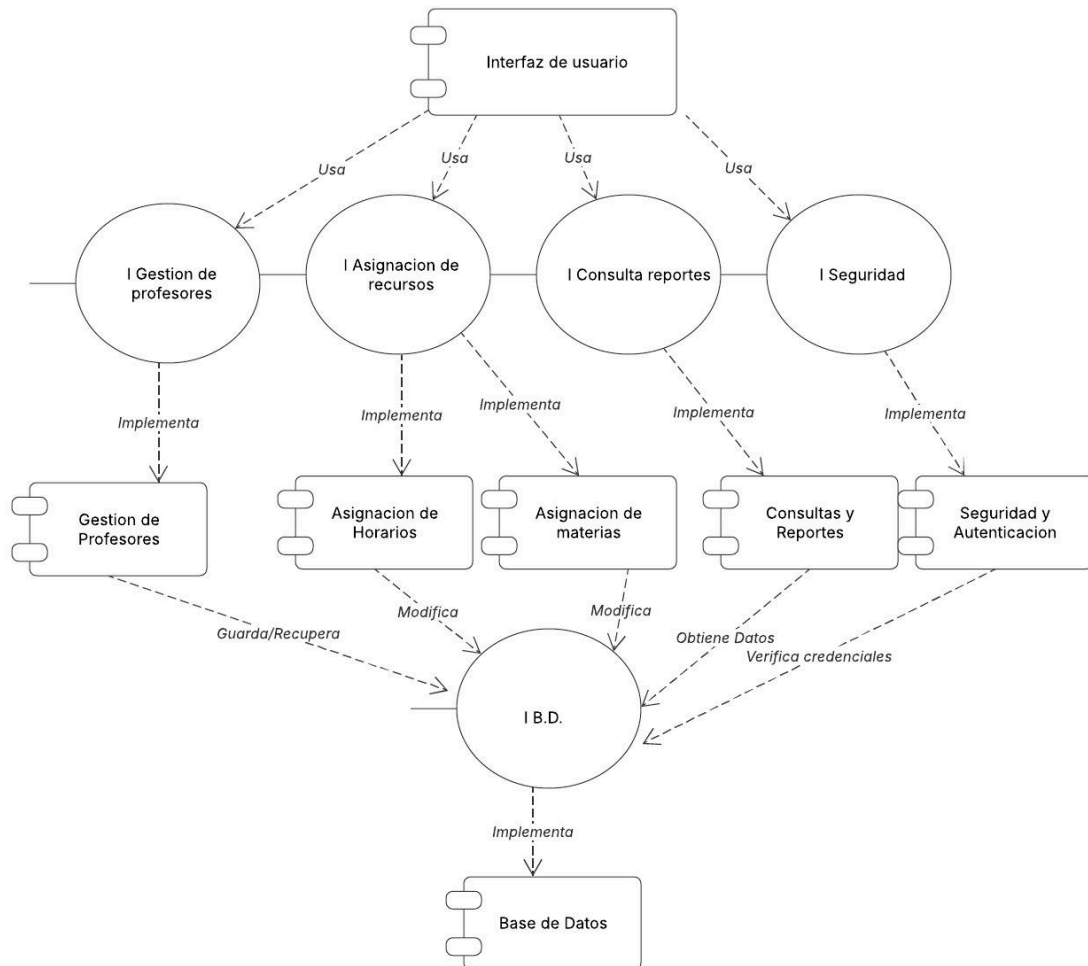
Este diagrama mostraría las dependencias entre los paquetes.

Versión	Fecha	Realizado por	Tiempo requerido
0.1	14/03/25	Cuapantecatl Ruiz Raul Irving	5:00 hrs

- Diagrama de Componentes UML:
 - Este diagrama detalla los componentes de software del sistema y sus interfaces. Se identificaron componentes como:
 - Componente Profesores: Responsable de las operaciones de administración de profesores.
 - Componente Materias: Responsable de las operaciones de gestión de materias.
 - Componente Horarios: Responsable de las operaciones de gestión de horarios.
 - Componente Consultas y Reportes: responsable de las operaciones de consulta de grupos de estudiantes, horarios y materias.
 - Componente Interfaz: Responsable de la interacción con los usuarios.
 - Componente BaseDatos: Responsable del acceso a la base de datos.
 - Componente Seguridad y Autenticación: Responsable del acceso al sistema.

Diagrama de componentes

Miguel Angel Garcia Leon | March 13, 2025

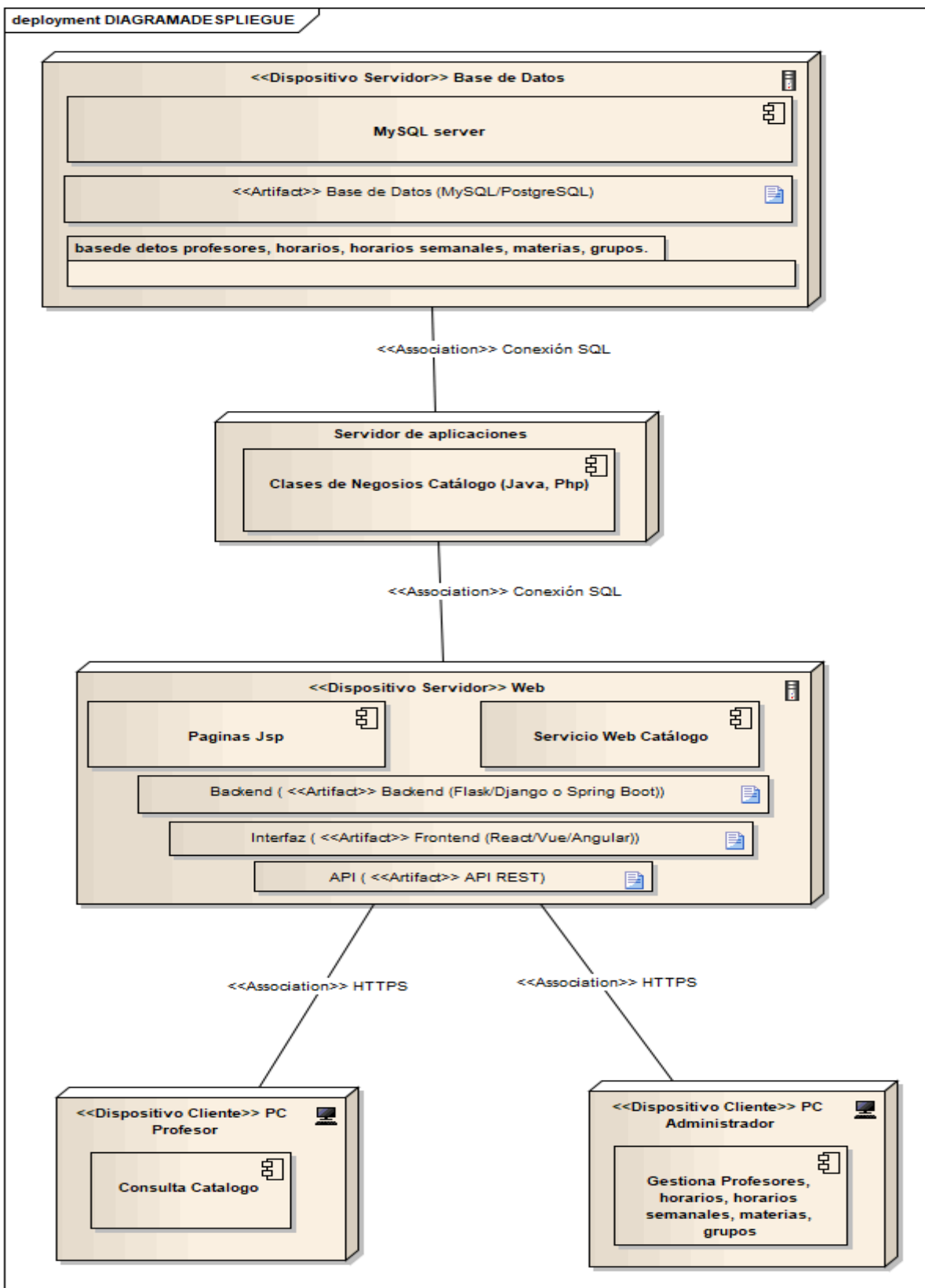


- Este diagrama mostraría las dependencias y las interfaces proporcionadas/requeridas por cada componente.
- Lenguajes de Descripción de Arquitecturas (ADL):
 - Estos lenguajes se podrían usar para describir formalmente la arquitectura del sistema, incluyendo los componentes, sus interfaces y las interacciones entre ellos.

3.2 Vista Física

La vista física se centra en la implementación y el despliegue del sistema en el entorno de ejecución.

- Diagrama de Despliegue UML:
 - Este diagrama mostrará los nodos físicos (servidores, dispositivos) donde se desplegarán los componentes del sistema. Se identificarán nodos como:
 - Servidor de Aplicaciones: Donde se desplegarán los componentes de la lógica de negocio.
 - Servidor de Base de Datos: Donde se desplegará la base de datos.
 - Estaciones de trabajo de los administradores y profesores.
 - Este diagrama mostraría la distribución de los componentes en los nodos y las conexiones de comunicación entre ellos.



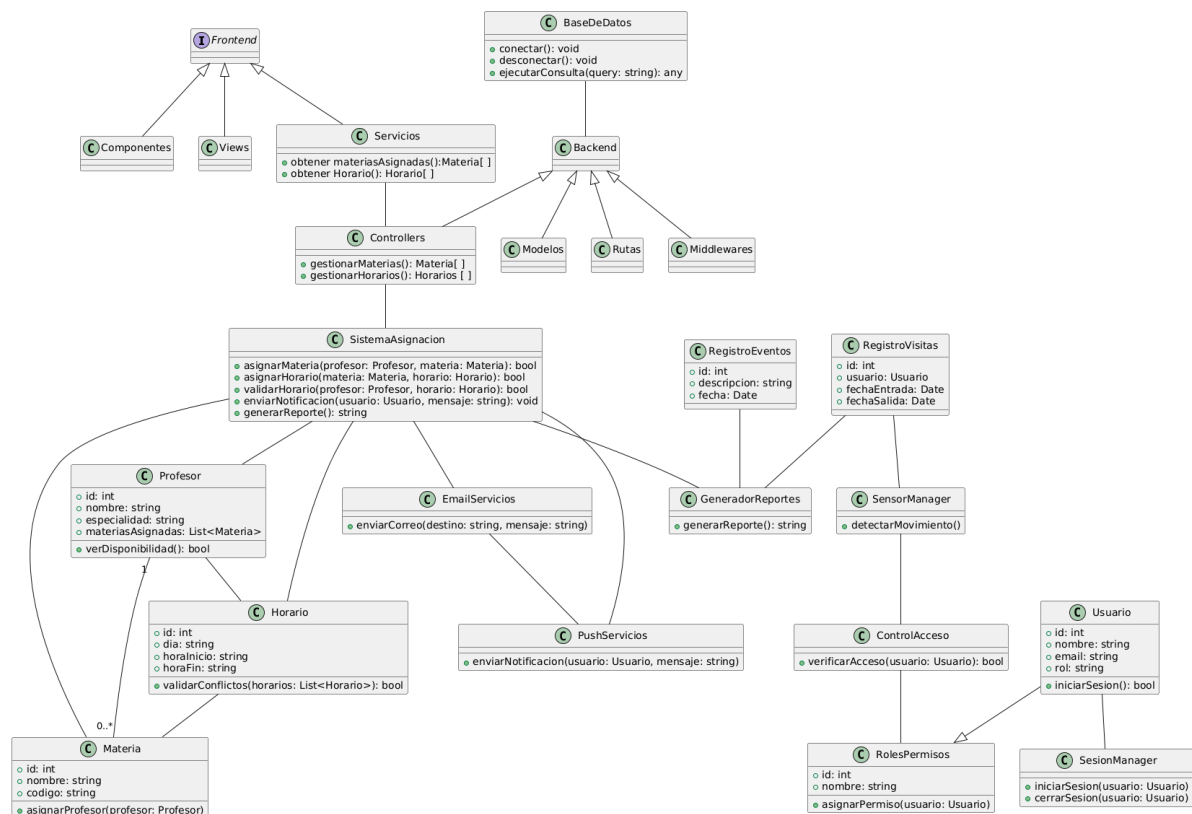
Versión	Fecha	Realizado por	Tiempo requerido
0.1	20/03/25	Cuapantecatl Ruiz Raul Irving	2:00 hrs

4. Logical del 5.4 IEEE 1016

Diagrama de Clases (UML Class Diagram)

El Diagrama de Clases representa la estructura estática del Sistema de Asignación de Horarios de Profesores, incluyendo las clases principales, sus atributos, métodos y relaciones entre ellas. Se han modelado clases como Profesor, Materia, Horario, Notificación y Reporte, estableciendo sus asociaciones mediante relaciones como herencia, composición y asociaciones directas.

- Punto de Vista de Composición: Modela la estructura interna del sistema, identificando sus principales componentes y relaciones mediante diagramas de clases UML.



Se implementó una mejora de asociación de la base de datos con la parte lógica del sistema

Versión	Fecha	Realizado por	Tiempo requerido
0.2	22/03/25	Garcia Leon Miguel Angel	2:00 hrs

Estos puntos de vista se utilizarán para garantizar que el diseño del sistema sea comprensible, estructurado y alineado con los requisitos funcionales y no funcionales previamente definidos.

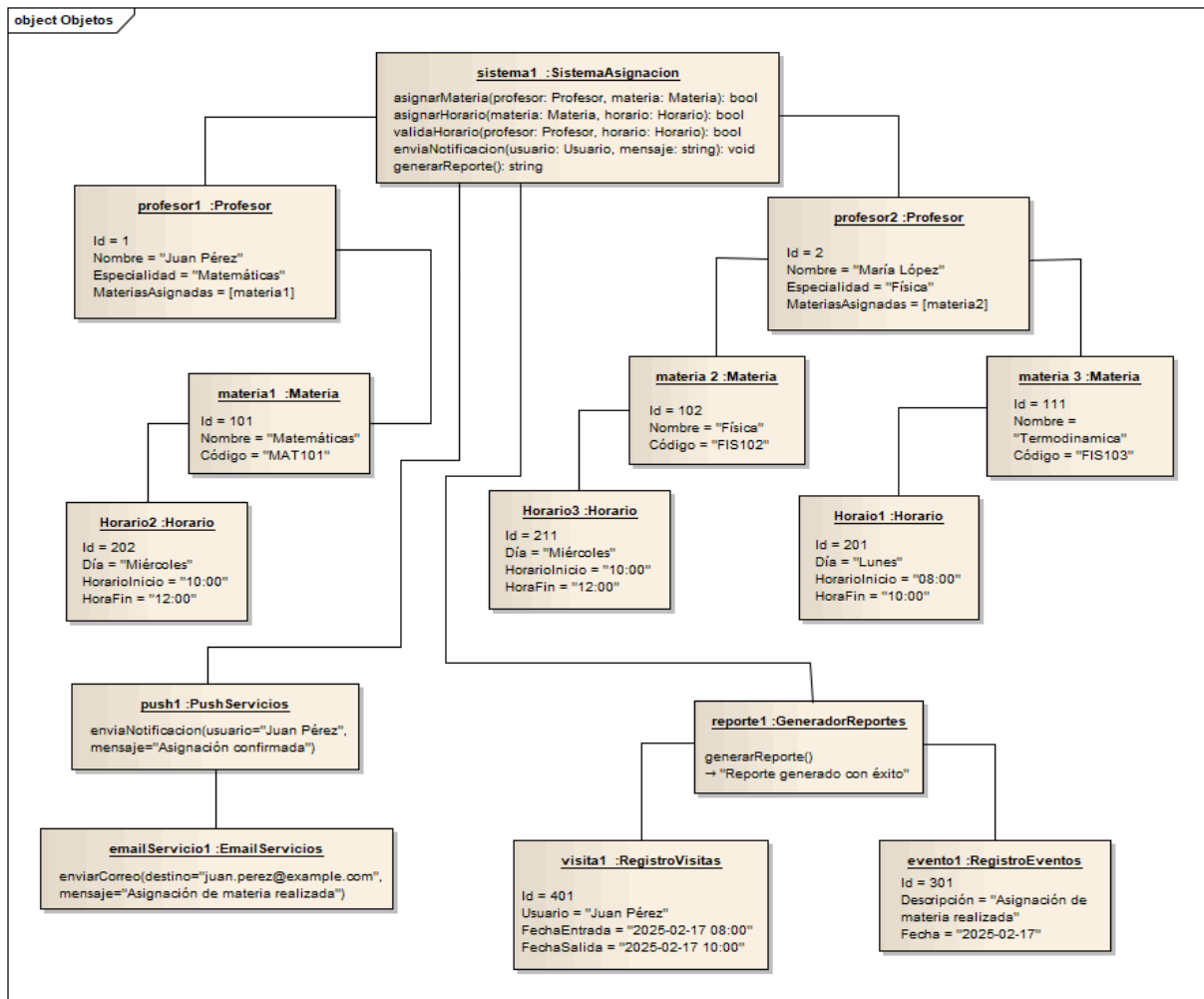
Diagrama de Objetos (UML Object Diagram)

El Diagrama de Objetos ejemplifica instancias concretas de las clases definidas en el Diagrama de Clases, mostrando un estado específico del sistema en ejecución.

Por ejemplo, se presentan instancias como Profesor: Juan Pérez, Materia: Matemáticas, Horario: Lunes 8:00 - 10:00, y Notificación: "Cambio de aula".

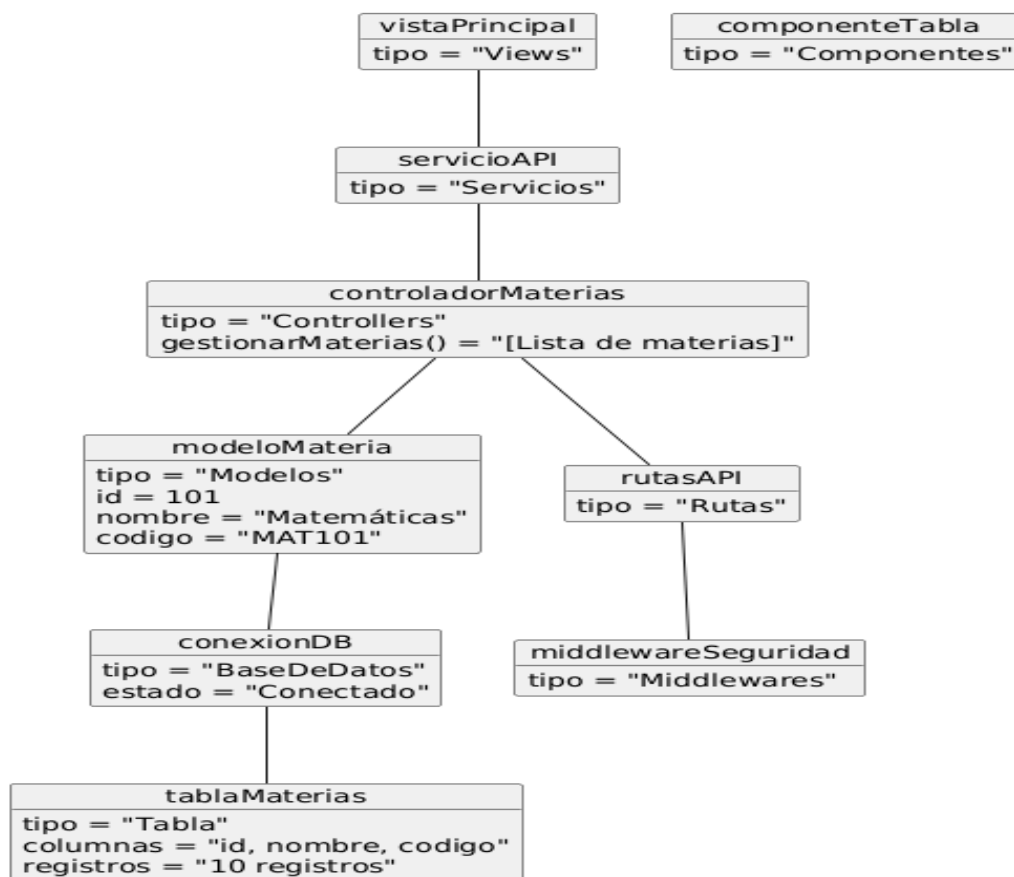
4.1 Relación con Logical (5.4)

- Estructura estática (class, interfaces, and their relationships): Muestra un escenario real de cómo los objetos interactúan dentro del sistema.
- Reuse of types and implementations (class, data types): Las instancias reutilizan estructuras definidas en el Diagrama de Clases, ejemplificando su uso práctico.



Versión	Fecha	Realizado por	Tiempo requerido
0.1	22/03/25	Cuapantecatl Ruiz Raul Irving	5:00 hrs

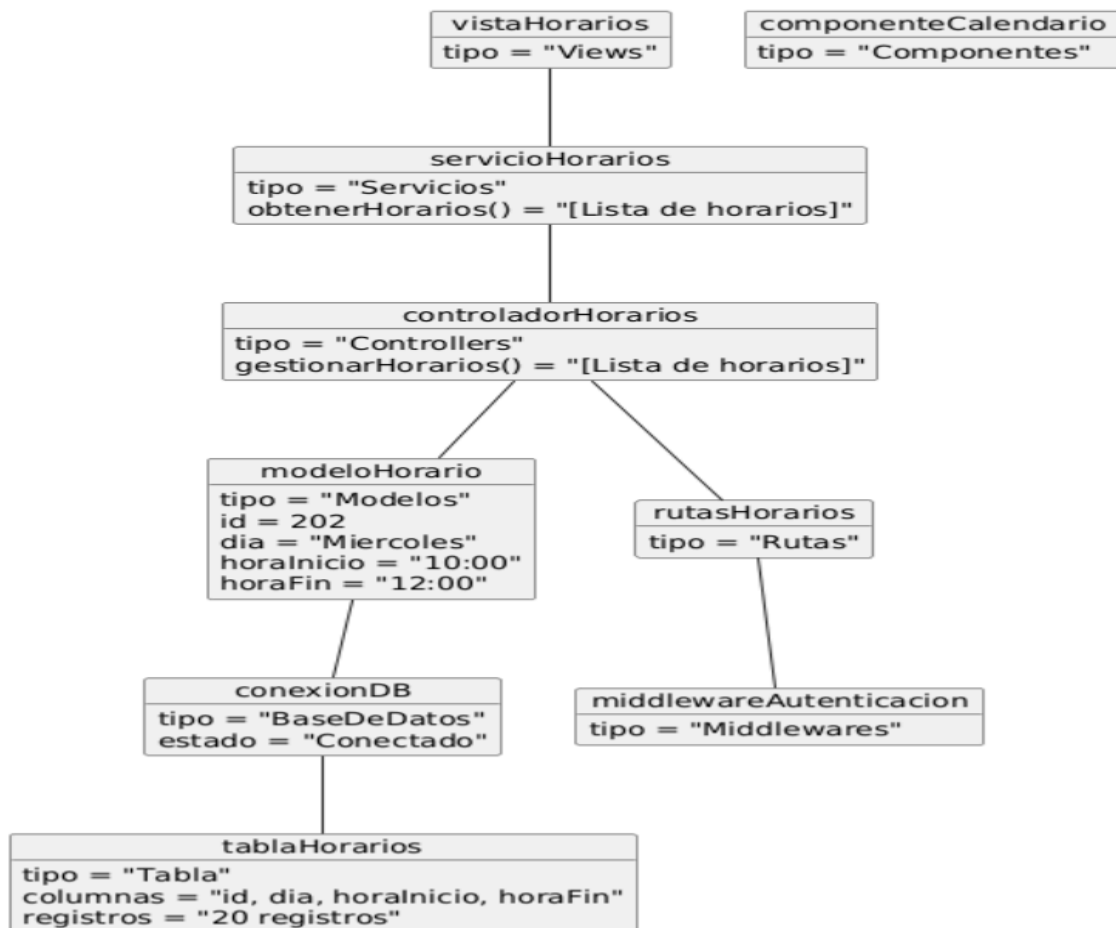
El diagrama de objetos para la gestión de materias representa cómo los diferentes elementos del sistema interactúan para asignar materias a los profesores. En este modelo:



Versión	Fecha	Realizado por	Tiempo requerido
0.1	23/03/25	García Leon Miguel Angel	3:00 hrs

- El Frontend cuenta con una vista específica para la gestión de materias y un servicio que obtiene la información desde el Backend.
- El Backend contiene un controlador encargado de gestionar las materias, las cuales se representan con objetos del modelo correspondiente.
- La Base de Datos almacena la información de las materias en una tabla específica y se gestiona a través de una conexión establecida.
- La comunicación entre estos elementos se realiza mediante rutas y middleware de autenticación, asegurando el acceso adecuado.

El diagrama de objetos de gestión de horarios se centra en cómo se administran y validan los horarios asignados a las materias y profesores. La estructura es similar al diagrama de materias, pero adaptada al manejo de horarios.



Versión	Fecha	Realizado por	Tiempo requerido
0.1	23/03/25	Garcia Leon Miguel Angel	3:00 hrs

- En el Frontend, la vista de horarios obtiene la información a través de un servicio y la representa gráficamente mediante un componente de calendario.
- En el Backend, el controlador de horarios administra la lógica de asignación, validación de conflictos y recuperación de datos desde el modelo correspondiente.

- La Base de Datos maneja los registros de horarios a través de una tabla que almacena los días, horas de inicio y fin, y posibles conflictos de asignación.
- Se mantiene una estructura segura mediante rutas protegidas por middleware de autenticación.

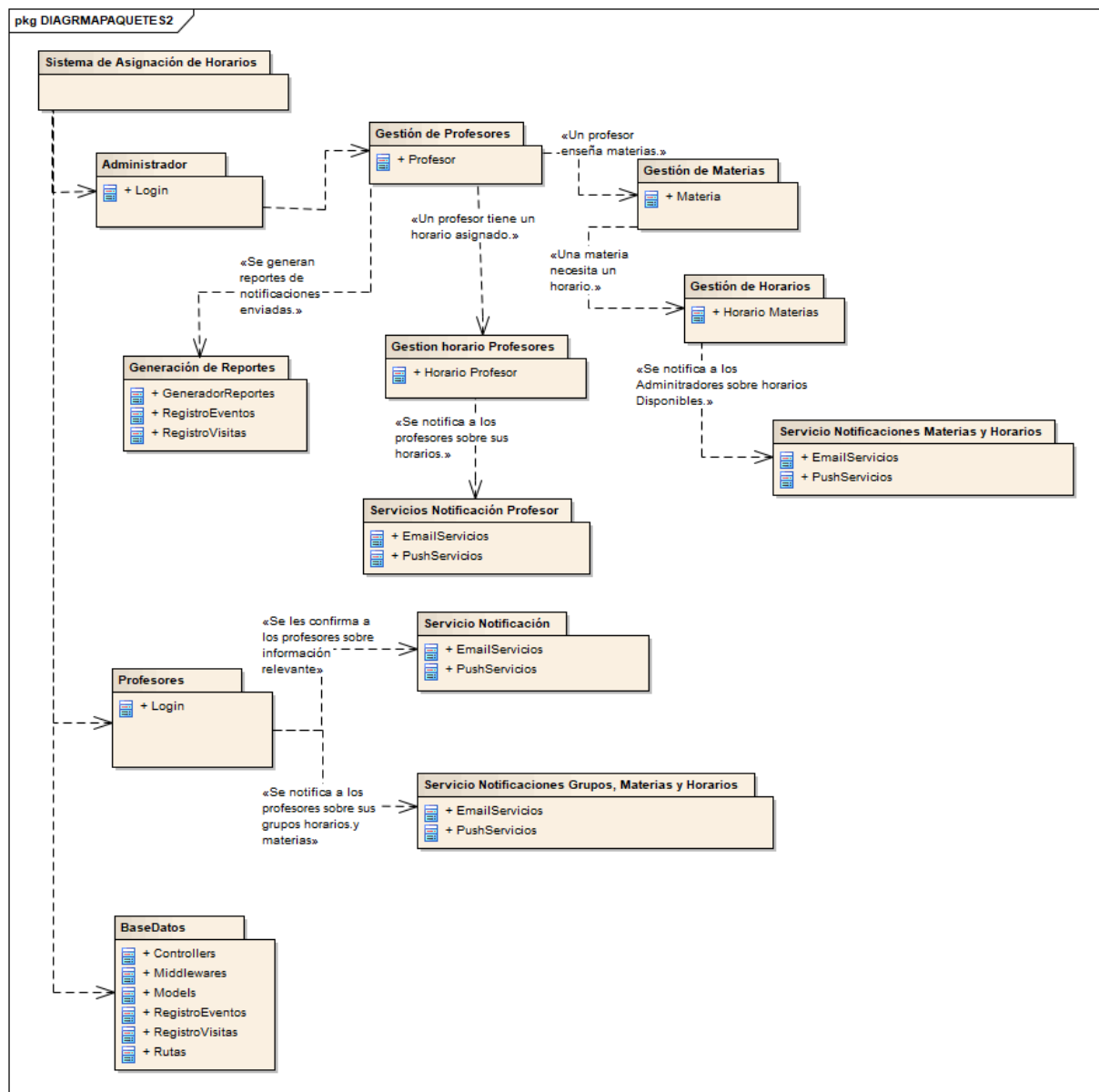
5. Dependency (5.5)

5.1 Diagrama de Paquetes (UML Package Diagram)

El Diagrama de Paquetes organiza el sistema en módulos lógicos y muestra sus dependencias.

Se han definido paquetes como Gestión de Profesores, Gestión de Materias, Gestión de Horarios, Servicios de Notificación y Generación de Reportes, estableciendo relaciones de dependencia entre ellos.

- Interconnection, sharing, and parameterization: Organiza el sistema en paquetes reutilizables y define las dependencias entre ellos.
- UML Package Diagram: Representa la agrupación lógica del sistema y su organización modular.



Versión	Fecha	Realizado por	Tiempo requerido
0.1	22/03/25	Cuapantecatl Ruiz Raul Irving	3:00 hrs

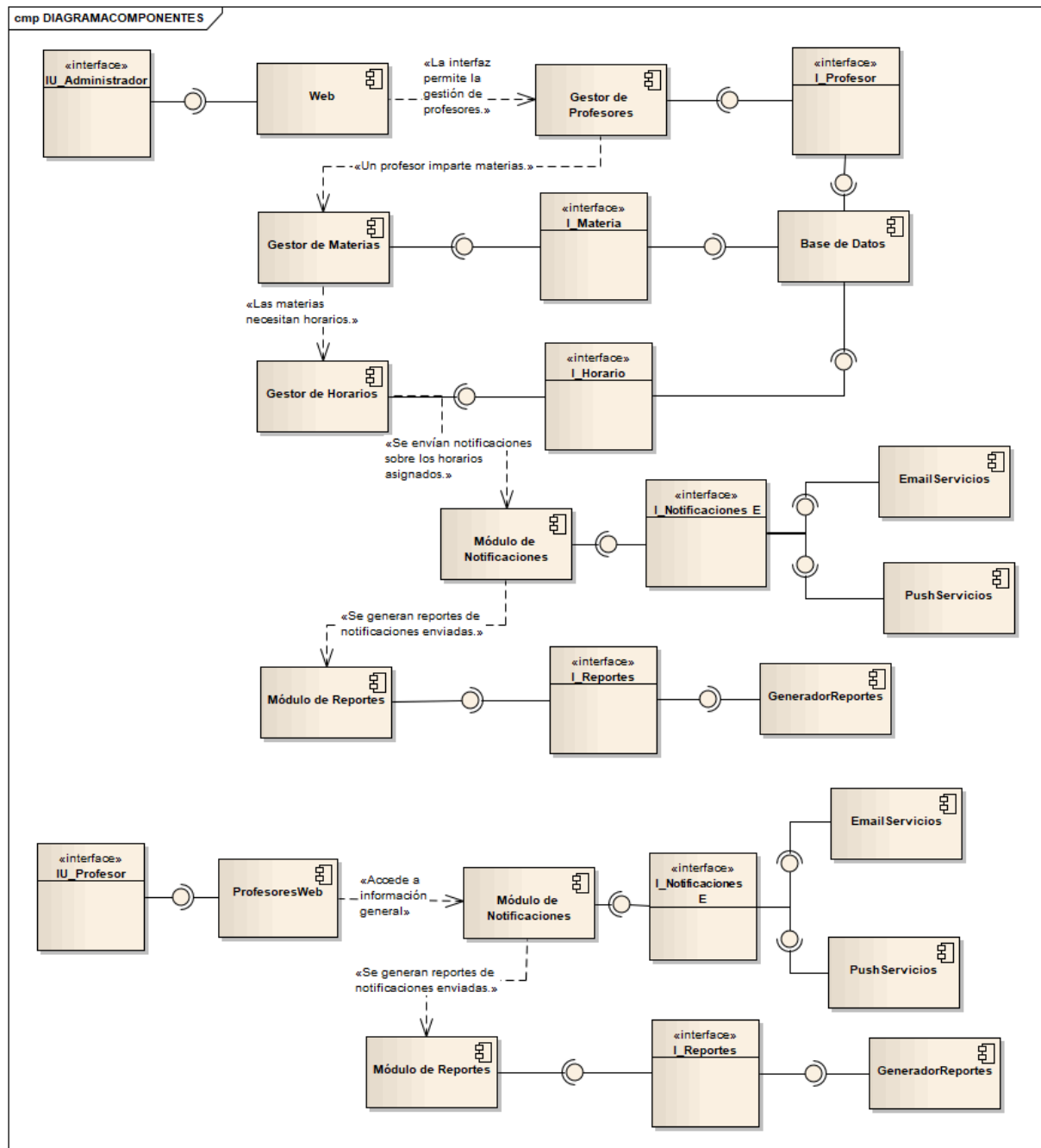
5.2 Diagrama de Componentes (UML Component Diagram)

El Diagrama de Componentes muestra la arquitectura física del sistema, definiendo los módulos independientes y sus interfaces de comunicación.

Se han modelado componentes como Interfaz Web, Gestor de Profesores, Gestor de Materias, Gestor de Horarios, Módulo de Notificaciones, Módulo de Reportes y Base de Datos, con sus respectivas interfaces y dependencias.

5.3 Relación con Dependency (5.5)

- Interconnection, sharing, and parameterization: Describe cómo los diferentes módulos del sistema están interconectados y comparten funcionalidades.
- UML Component Diagram: Representa la arquitectura física y los servicios que cada componente proporciona a otros.



Versión	Fecha	Realizado por	Tiempo requerido
0.1	22/03/25	Cuapantecatl Ruiz Raul Irving	4:00 hrs

6. Patterns (5.7)

6.1 Patrón Strategy (Estrategia)

El Patrón Strategy permite definir un conjunto de algoritmos o estrategias para la asignación de horarios de profesores y seleccionarlos dinámicamente según las necesidades del sistema.

Implementación en el Sistema:

Este patrón se aplicará en la funcionalidad de asignación de horarios, donde diferentes estrategias pueden ser utilizadas según criterios específicos como:

- Disponibilidad del profesor
- Carga horaria máxima permitida
- Preferencias de los profesores
- Restricciones institucionales

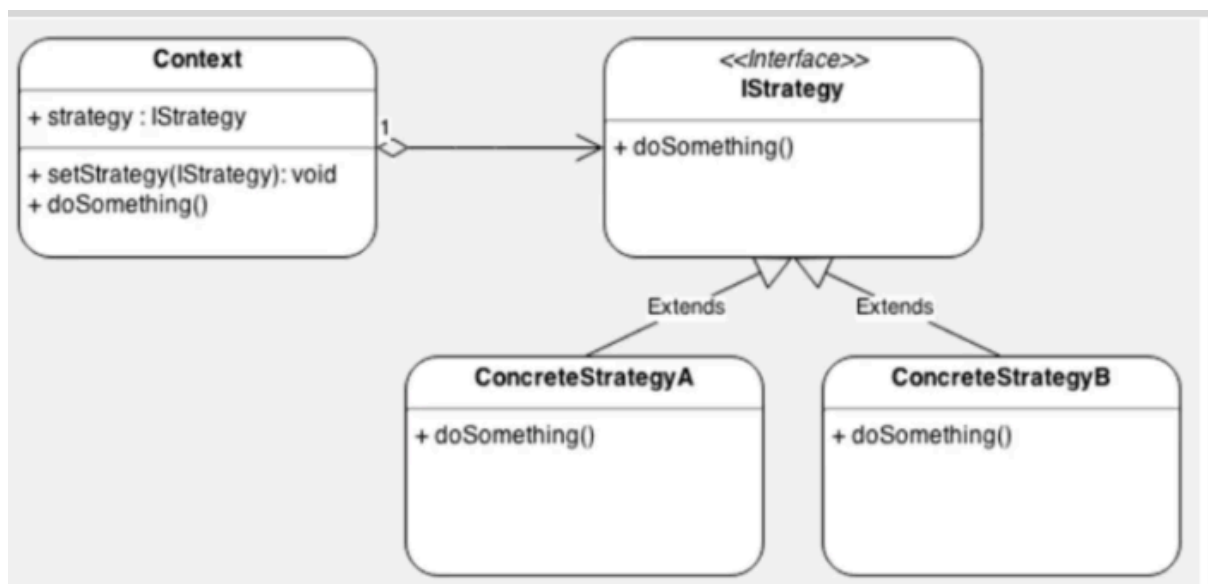


Imagen 1.1

Cada estrategia se implementará como una clase concreta que sigue una interfaz común, permitiendo su intercambio sin modificar la estructura del sistema.

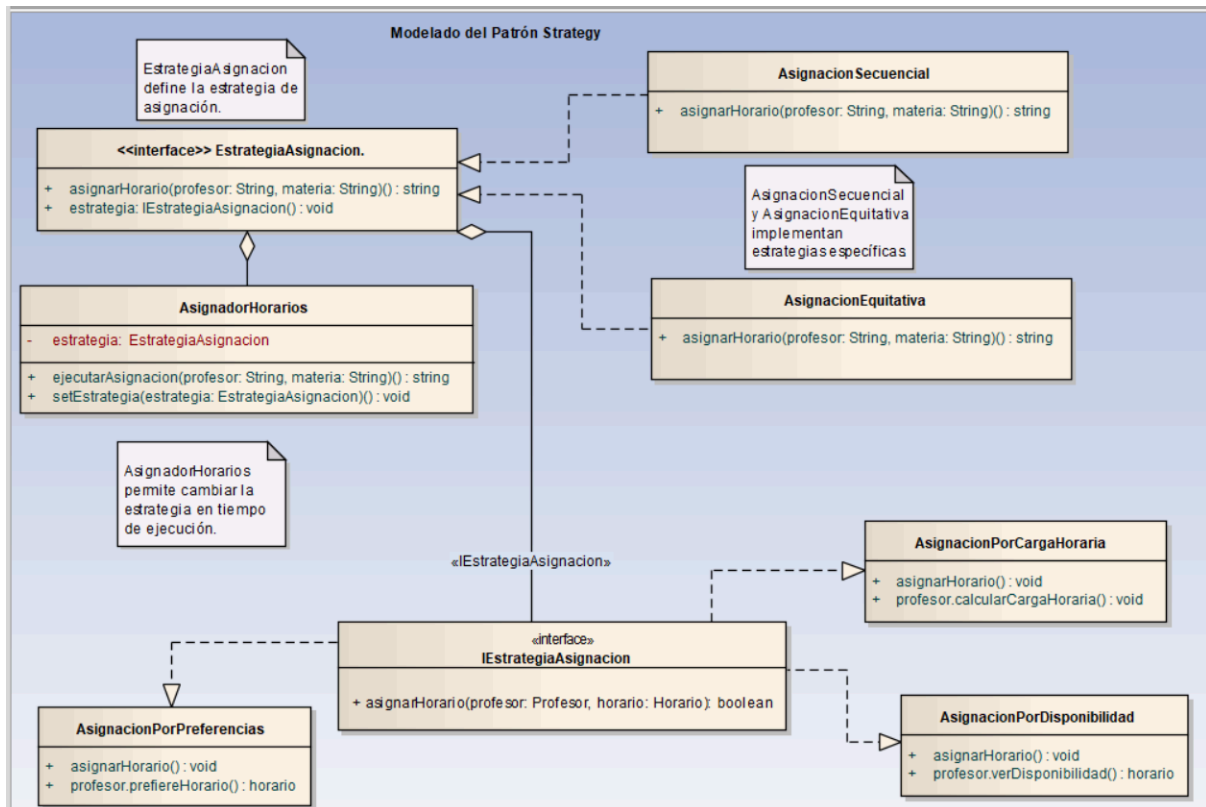


Imagen 1.2

Clase	Método	Descripción
AsignacionPorDisponibilidad	asignarHorario()	Verifica disponibilidad del profesor.
AsignacionPorCargaHoraria	asignarHorario()	Controla que no exceda la carga máxima.
AsignacionPorPreferencias	asignarHorario()	Verifica si el horario coincide con las preferencias.

Beneficios

- Permite definir múltiples estrategias de asignación.
- Mejora la flexibilidad del código al evitar estructuras rígidas.
- Facilita la integración de nuevas reglas sin modificar el núcleo del sistema
- Flexibilidad: Se pueden cambiar estrategias sin afectar el código existente.
- Escalabilidad: Permite agregar nuevas estrategias sin modificar las clases principales.
- Mantenimiento Simplificado: Evita grandes modificaciones en la lógica central.

- Personalización: Adapta la asignación de horarios según reglas específicas de la institución.

6.2 Patrón Observer (Observador)

El Patrón Observer permite que los objetos interesados en un evento sean notificados automáticamente cuando ocurre un cambio en el estado del sistema.

Implementación en el Sistema

Se aplicará en la gestión de notificaciones cuando se realicen cambios en los horarios asignados. Ejemplo de uso:

- Si el horario cambia, los profesores y alumnos registrados recibirán una notificación.
- Se integrará con los servicios de EmailServicios y PushServicios para enviar notificaciones en tiempo real.

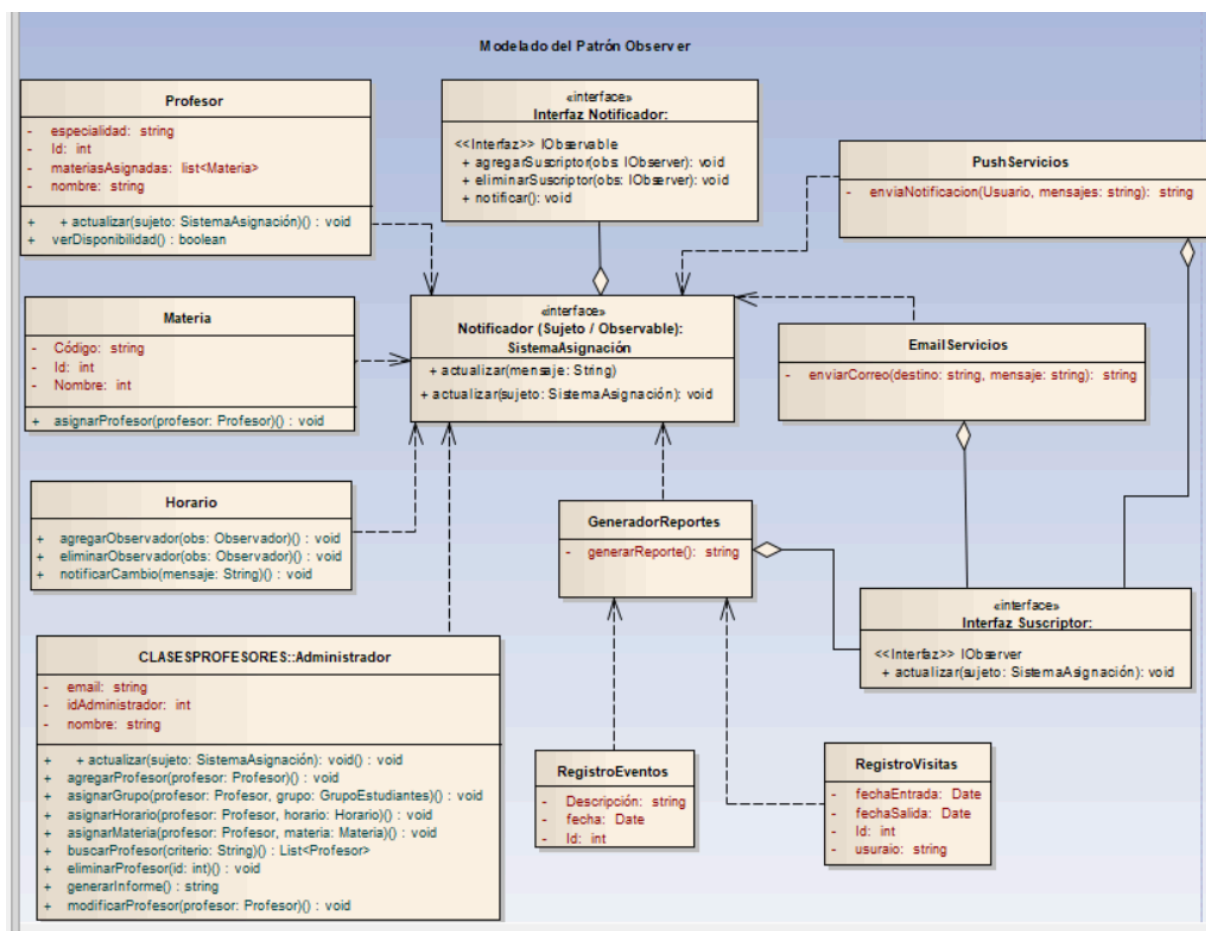


Imagen1.3

Lógica del patrón Observer aplicada:

Evento: asignarMateria o asignarHorario

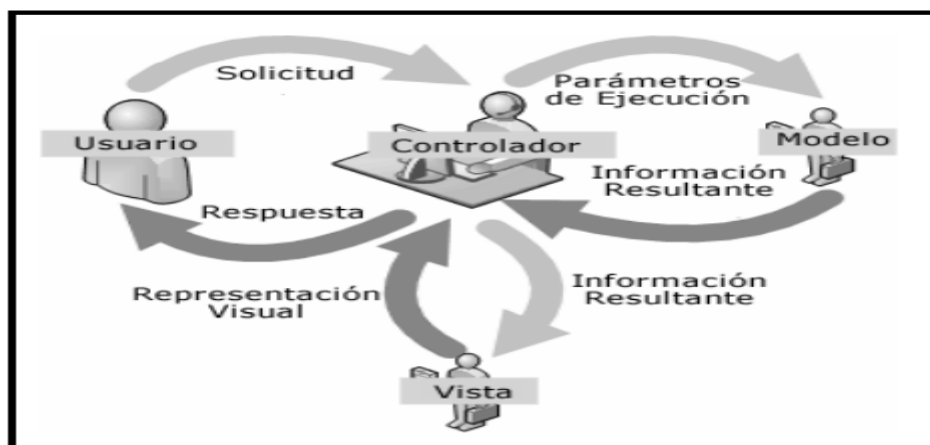
1. SistemaAsignacion actualiza los datos.
2. Llama a notificar("Materia asignada a profesor X").
3. Cada suscriptor (EmailServicios, PushServicios, GeneradorReportes) reacciona vía actualizar.

Beneficios

- Facilita la comunicación automática de cambios en los horarios.
- Reduce la dependencia entre módulos mediante una arquitectura desacoplada.
- Mejora la escalabilidad del sistema al permitir nuevos suscriptores sin modificar el código base.

6.3 Modelo-Vista-Controlador (MVC):

Un patrón que separa la aplicación en tres componentes interconectados: Modelo (datos y lógica de negocios), Vista (interfaz de usuario) y Controlador (maneja la entrada del usuario y actualiza el modelo y la vista). Componentes clave: Modelo, Vista, Controlador.



Implementación en el Sistema

El sistema de asignación de horarios utilizará el patrón **MVC** para separar la lógica de negocio de la presentación y la gestión de eventos.

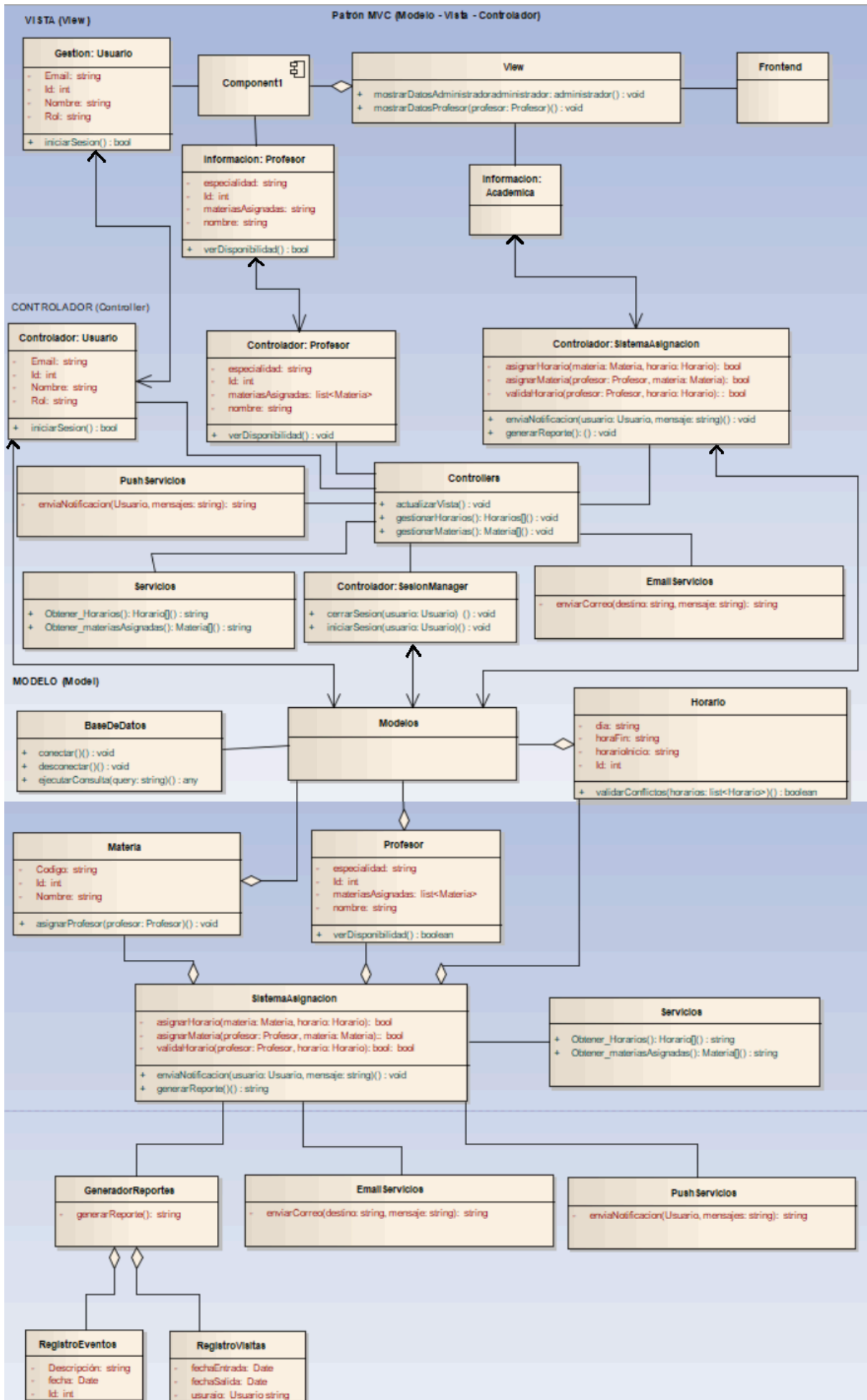


Imagen 1.4

Versión	Fecha	Realizado por	Tiempo requerido
Imagenes {1.1, 1.2, 1.3, 1.4} 0.1	11/04/25	Cuapantecatl Ruiz Raul Irving	8:00 hrs

El controlador se encarga de la comunicación entre la vista y el modelo, evitando dependencias directas entre ellos.

Beneficios

- Modularidad: Separa la lógica de negocio de la interfaz de usuario.
- Facilita el mantenimiento: Cambios en la vista no afectan al modelo y viceversa.
- Reutilización: Permite reutilizar el modelo en diferentes interfaces (Web, móvil, escritorio).

En resumen de Patrones en el Sistema.

Patrón	Descripción	Implementación	Beneficios
Strategy	Define estrategias dinámicas para la asignación de horarios.	Diferentes clases representan reglas de asignación.	Flexibilidad, facilidad de mantenimiento.
Observer	Notifica automáticamente cambios en horarios.	Profesores y alumnos son observadores del horario.	Desacoplamiento, escalabilidad.
MVC	Separa lógica de negocio, interfaz y controladores.	Modelo (datos), Vista (UI), Controlador (interacción).	Modularidad, reutilización, fácil mantenimiento.

Estos patrones permiten que el Sistema de Asignación de Horarios de Profesores sea escalable, flexible y fácil de mantener. Su implementación facilita la gestión eficiente de asignaciones, notificaciones y la interacción con el usuario.

7. Interface (5.8)

Basado en los casos de uso y requisitos funcionales (RF) del Sistema de Asignación de Horarios, para darles solución e implementación se utilizará:

- MySQL Workbench (como base de datos),
- Servidor Django (como backend),
- Aplicación Web (como interfaz de usuario),
- Y los actores ya mencionados: Administrador y Profesor.

7.1 Lenguaje de Definición de Interfaces (IDL)

Este sistema está diseñado con arquitectura web basada en el modelo cliente-servidor, por lo que las interfaces entre los componentes siguen estándares modernos como lo son:

- RESTful API: para comunicación entre el frontend web y el backend (servidor Django).
- ORM de Django: actúa como interfaz hacia la base de datos MySQL.
- JSON: formato de datos para intercambios entre frontend y backend.
- HTML/CSS/JavaScript: lenguajes para la interfaz visual de usuario.

Cada una de estas interfaces expone funciones específicas que están alineadas con los casos de uso y requisitos funcionales (RF) del sistema.

El sistema de asignación de horarios utiliza un enfoque basado en Lenguajes de Definición de Interfaz (IDL, por sus siglas en inglés) para estandarizar la comunicación entre los componentes del sistema, así como su interacción con sistemas externos. Un IDL permite especificar de forma clara y estructurada las operaciones que están disponibles en cada módulo o componente, facilitando así la interoperabilidad, independientemente del lenguaje de programación subyacente.

Cada componente funcional del sistema tiene definida una interfaz clara y separada que especifica las operaciones que puede realizar, y cómo interactúa tanto con los actores del sistema como con otros servicios o recursos externos, tales como la base de datos MySQL y el servidor backend (Django). Estas interfaces son fundamentales para permitir la implementación modular y escalable del sistema.

Lenguajes de definición de interfaz (IDL), diagrama de componentes UML

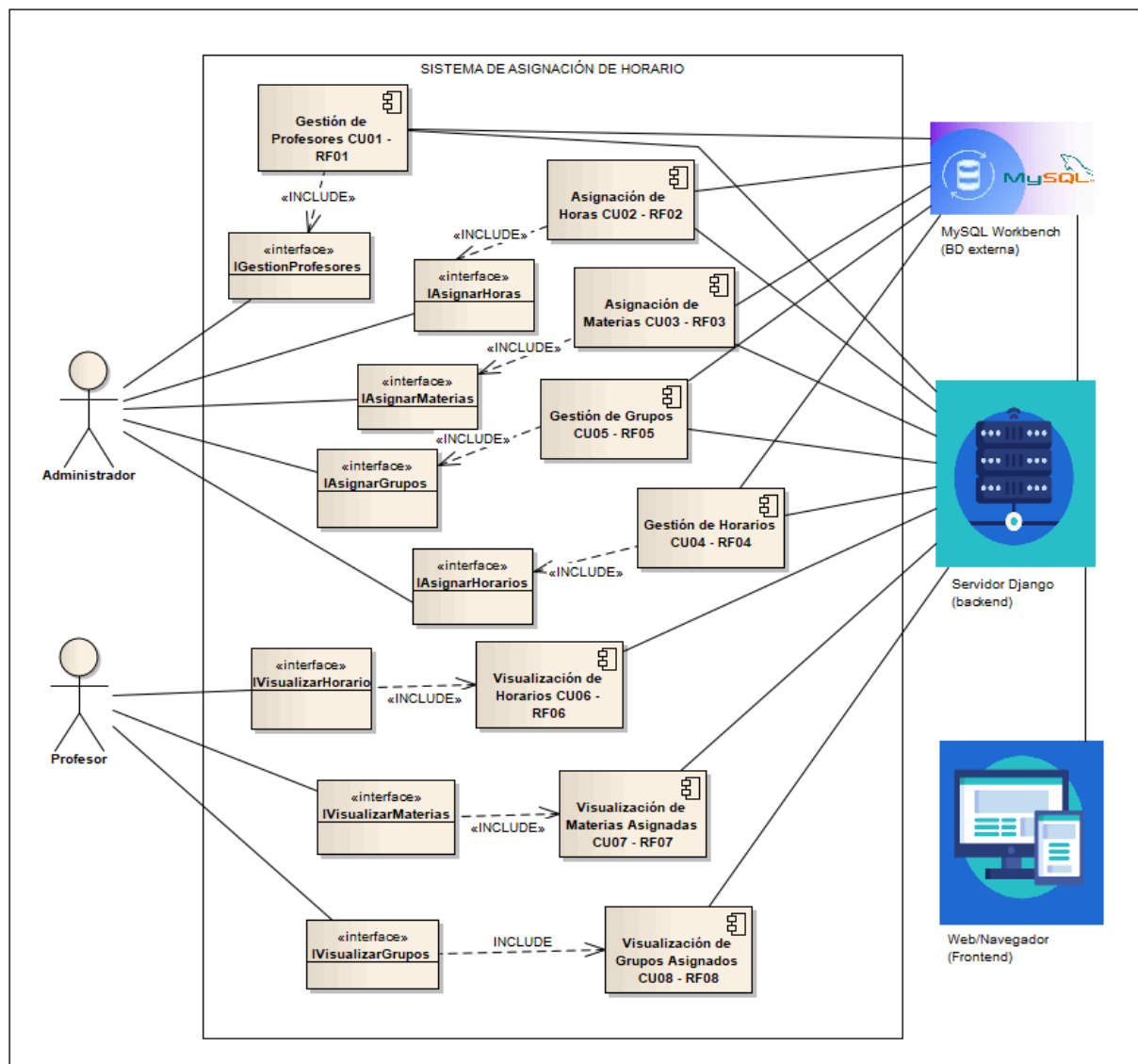


Imagen 5.8

Versión	Fecha	Realizado por	Tiempo requerido
Imagenes 5.8	30/04/25	Cuapantecatli Ruiz Raul Irving	3:00 hrs

Descripción del Diagrama de Componentes del Sistema de Asignación de Horarios:

El diagrama de componentes del Sistema de Asignación de Horarios se representa mediante un rectángulo principal titulado "Sistema de Asignación de Horarios". Dentro de este recuadro, se identifican los siguientes componentes, cada uno asociado a uno o varios casos de uso.

- **Gestión de Profesores (CU01, RF01):** Responsable de la funcionalidad para agregar, modificar o eliminar profesores.

- Asignación de Horas (CU02, RF02): Encargado de permitir la asignación de horas semanales a los profesores.
- Asignación de Materias (CU03, RF03): Gestiona la asignación de materias a los profesores según su especialización.
- Gestión de Grupos (CU05, RF05): Responsable de la asignación de grupos de estudiantes a los profesores para cada materia.
- Gestión de Horarios (CU04, RF04): Permite establecer los horarios de cada materia para los profesores.
- Visualización de Horarios (CU06, RF06): Proporciona la funcionalidad para que los profesores vean su horario diario y semanal.
- Visualización de Materias Asignadas (CU07, RF07): Permite a los profesores ver las materias que deben impartir.
- Visualización de Grupos Asignados (CU08, RF08): Facilita que los profesores conozcan los grupos de estudiantes a su cargo.

Dentro del mismo recuadro del sistema, también se definen las interfaces que estos componentes exponen para interactuar con otros componentes o actores:

- Administrador: Interactúa con las interfaces de gestión y asignación (Gestión de Profesores, Asignación Horas, Asignación de Materias, Asignación de Grupos, Asignación de Horarios).
- Profesor: Interactúa principalmente con las interfaces de visualización (Visualizar Horarios, Visualizar Materias, Visualizar Grupos).

Fuera del recuadro del sistema, se identifican los actores que interactúan con el sistema:

Además, se representan los sistemas externos con los que el Sistema de Asignación de Horarios tiene comunicación:

- MySQL Workbench (Base de Datos Externa): Representa la base de datos donde se almacenan los datos del sistema.
- Servidor de Django (Backend): Actúa como la capa de lógica de negocio y proporciona los servicios al frontend.
- Web Navegador (Frontend): Es la interfaz a través de la cual los actores interactúan con el sistema.

Estos tres elementos externos se muestran relacionados entre sí, indicando su colaboración en la funcionalidad del sistema.

Explicación de las Asociaciones o Relaciones:

- El Administrador tiene una asociación con las interfaces de Gestión de Profesores, Asignación de Materias, Asignación de Grupos y Asignación de Horarios, lo que indica que el administrador utiliza estas interfaces para realizar las tareas de gestión del sistema.
- El Profesor tiene una asociación con las interfaces de Visualizar Horarios, Visualizar Materias y Visualizar Grupos, lo que significa que el profesor utiliza estas interfaces para consultar la información relevante para él.
- El sistema MySQL Workbench (Base de Datos Externa) tiene una asociación con los componentes internos de Gestión de Profesores (CU01), Asignación de Horas (CU02), Asignación de Materias (CU03), Gestión de Horarios (CU04) y Gestión de Grupos (CU05). Esto indica que estos componentes acceden y manipulan los datos almacenados en la base de datos para persistir la información gestionada por el administrador.
- El Servidor de Django tiene una asociación con todos los ocho componentes (CU01 hasta CU08). Esto sugiere que el backend implementado en Django contiene la lógica de negocio para todos los casos de uso del sistema, actuando como el motor que orquesta la funcionalidad. La comunicación entre el frontend y el backend se realiza a través de las interfaces expuestas por el servidor de Django.
- El Web Navegador (Frontend) interactúa con el Servidor de Django, enviando peticiones y recibiendo respuestas para presentar la interfaz de usuario a los actores.

8. Structure (5.9): Componentes y Organización Interna del Sistema de Asignación de Horarios

El sistema de asignación de horarios se organiza en base a los principales sujetos de diseño, componentes y las clases que los conforman. El sistema se concibe como una arquitectura modular, donde está compuesto por diferentes módulos que interactúan para garantizar el funcionamiento correcto del proceso de asignación,

gestión y visualización de horarios académicos. A nivel de estructura, se identifican los siguientes elementos clave:

8.1 Sujetos de diseño

Los sujetos de diseño son las entidades principales que representan responsabilidades dentro del sistema. En este caso, se tienen:

1. Administrador

Es el actor principal encargado de la gestión completa del sistema. Administra usuarios, períodos académicos, asignaturas, grupos y horarios.

2. Docente

Es el usuario que consulta los horarios asignados y puede enviar solicitudes o restricciones de disponibilidad.

3. Sistema

Actúa como un sujeto automatizado encargado de realizar validaciones, verificar conflictos y asignar horarios.

8.2 Componentes del sistema

Los componentes representan agrupaciones funcionales dentro del diseño del software. Para este sistema, los componentes más relevantes son:

1. Componente de Autenticación

Maneja el inicio de sesión, validación de credenciales y control de accesos dependiendo del tipo de usuario (administrador o docente).

2. Componente de Gestión Académica

Permite la administración de asignaturas, grupos, docentes, aulas y periodos académicos.

3. Componente de Asignación de Horarios

Se encarga de generar, modificar y validar los horarios. Incluye reglas para evitar conflictos de tiempo, disponibilidad de aulas y disponibilidad de

docentes.

4. Componente de Visualización y Reportes

Permite generar vistas organizadas por docente, grupo o aula, y la exportación o impresión de los horarios.

5. Componente de Base de Datos

Gestiona el almacenamiento de toda la información relacionada con usuarios, materias, horarios, periodos, aulas y configuraciones del sistema.

Estos componentes se organizan en capas lógicas (presentación, lógica de negocio, datos) siguiendo el patrón MVC para separar las responsabilidades y facilitar el desarrollo y mantenimiento del sistema.

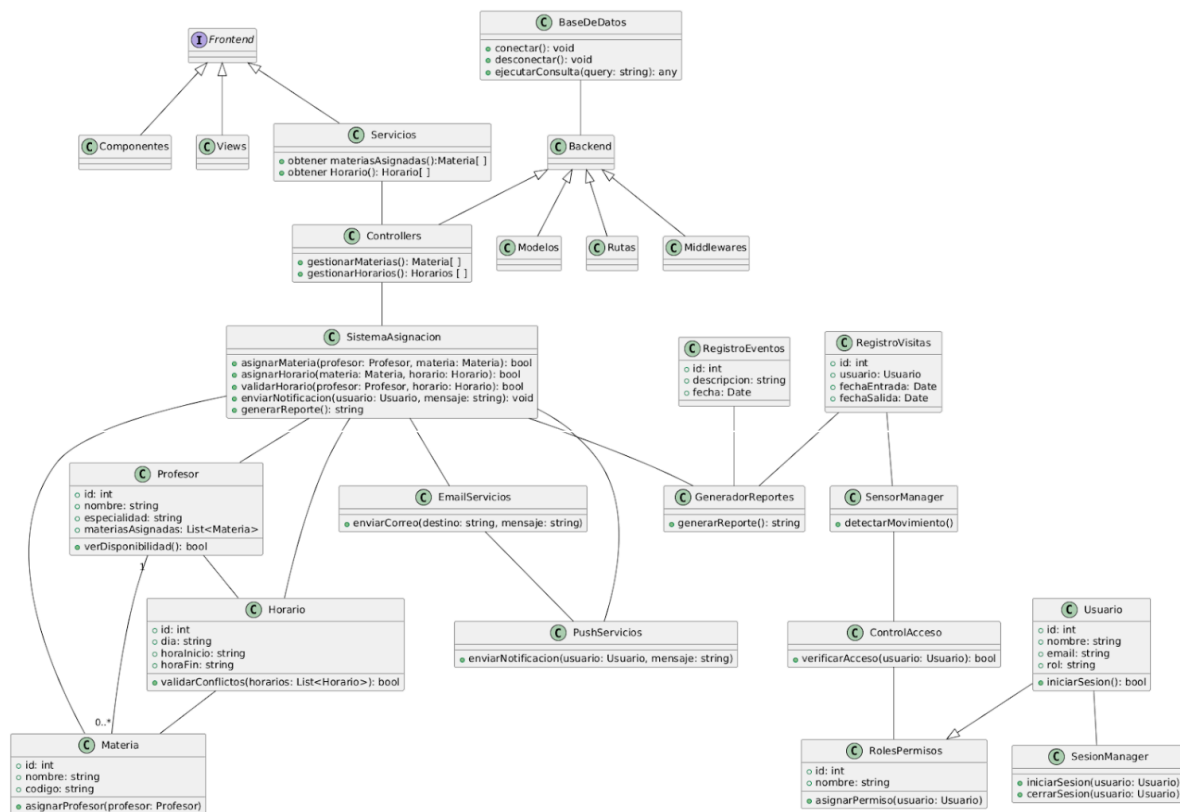
Organización General del Sistema

La clase principal del sistema es SistemaAsignación, la cual orquesta las operaciones esenciales como la asignación de materias y horarios, validación de disponibilidad de profesores, generación de reportes y envío de notificaciones.

8.3 Asociación de componentes principales:

- SistemaAsignación está asociada directamente con las clases: Profesor, Materia, Horario, EmailServicios, PushServicios y GeneradorReportes.
- Además, SistemaAsignación se relaciona con el Control, que a su vez gestiona a Servicios y a Backend, formando el núcleo de la lógica operativa del sistema.

DIAGRAMA DE CLASES:



8.4 Diagrama de Estructura (Clases UML)

El diagrama de clases del sistema refleja la interacción entre estas clases, su jerarquía (mediante generalizaciones) y sus asociaciones. La clase central SistemaAsignación se conecta a la lógica de negocio a través de Control, que distribuye la funcionalidad entre Servicios y Backend. El Frontend facilita la interacción con los usuarios y se construye sobre Componentes y Views.

8.5 División Arquitectónica

1. Backend

El backend es el motor de procesamiento del sistema y está compuesto por los siguientes elementos:

- Modelos: Representan las estructuras de datos persistentes en la base de datos.
- Rutas: Manejan el direccionamiento de solicitudes HTTP hacia los controladores correspondientes.

- Middlewares: Gestionan la lógica intermedia (autenticación, validación de entrada, etc.).
- BaseDeDatos: Clase encargada de establecer conexión con la base de datos externa, ejecutar consultas y desconectarse.

Todas estas clases están generalizadas bajo la clase Backend.

2. Controladores y Servicios

- Controllers: Administra la lógica de negocio básica, incluyendo la gestión de materias y horarios.
- Servicios: Interactúan con el controlador para obtener materias asignadas y horarios disponibles.

Ambas clases son gestionadas por la clase Control.

3. Frontend

El frontend está compuesto por:

- Frontend: Punto de entrada visual del sistema.
- Componentes y Views: Representan los elementos visuales reutilizables e interfaces gráficas.

Todos estos están relacionados mediante generalizaciones y forman parte del flujo de usuario.

8.6 Clases Funcionales Clave

Clase Profesor: Encargada de representar a un docente, permite consultar su disponibilidad y gestionar sus materias asignadas. Se relaciona con Horario y Materia.

Clase Materia: Define el contenido académico y permite su asignación a un profesor.

Clase Horario: Administra los bloques de tiempo y valida conflictos con horarios ya asignados. Tiene relación tanto con Materia como con Profesor.

Clase EmailServicios y PushServicios: Encargadas del envío de notificaciones por correo o sistema push. PushServicios está relacionada tanto con EmailServicios como con SistemaAsignación.

Clase GeneradorReportes: Produce los reportes administrativos y académicos, y está asociada a las clases RegistroEventos y RegistroVisitas.

Clase SensorManager y ControlAcceso: Permiten gestionar y registrar el acceso al sistema, detectando movimiento y validando permisos.

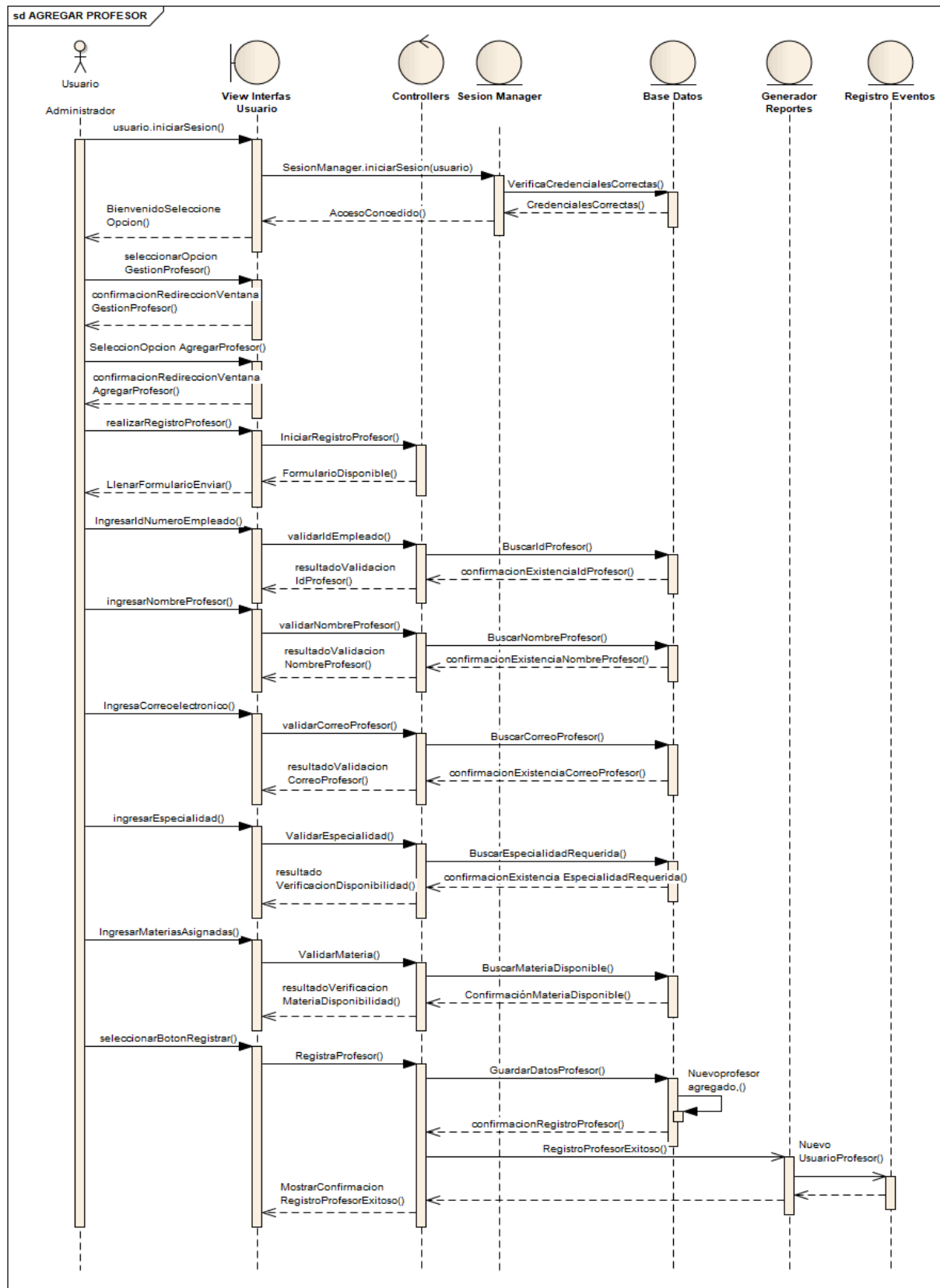
Clase Usuario: Clase base para todos los actores del sistema. Está asociada a SesionManager, RolesPermisos y otras entidades de autenticación.

Seguridad y Sesión

- RolesPermisos determina los accesos permitidos a través del rol de usuario, generalizado desde Usuario.
- SesionManager administra el inicio y cierre de sesiones, asegurando que solo los usuarios autorizados accedan al sistema.

9. Interacción (5.10)

9.1 DIAGRAMA DE SECUENCIA 1 DEL CASO DE USO 01 AGREGAR PROFESOR:



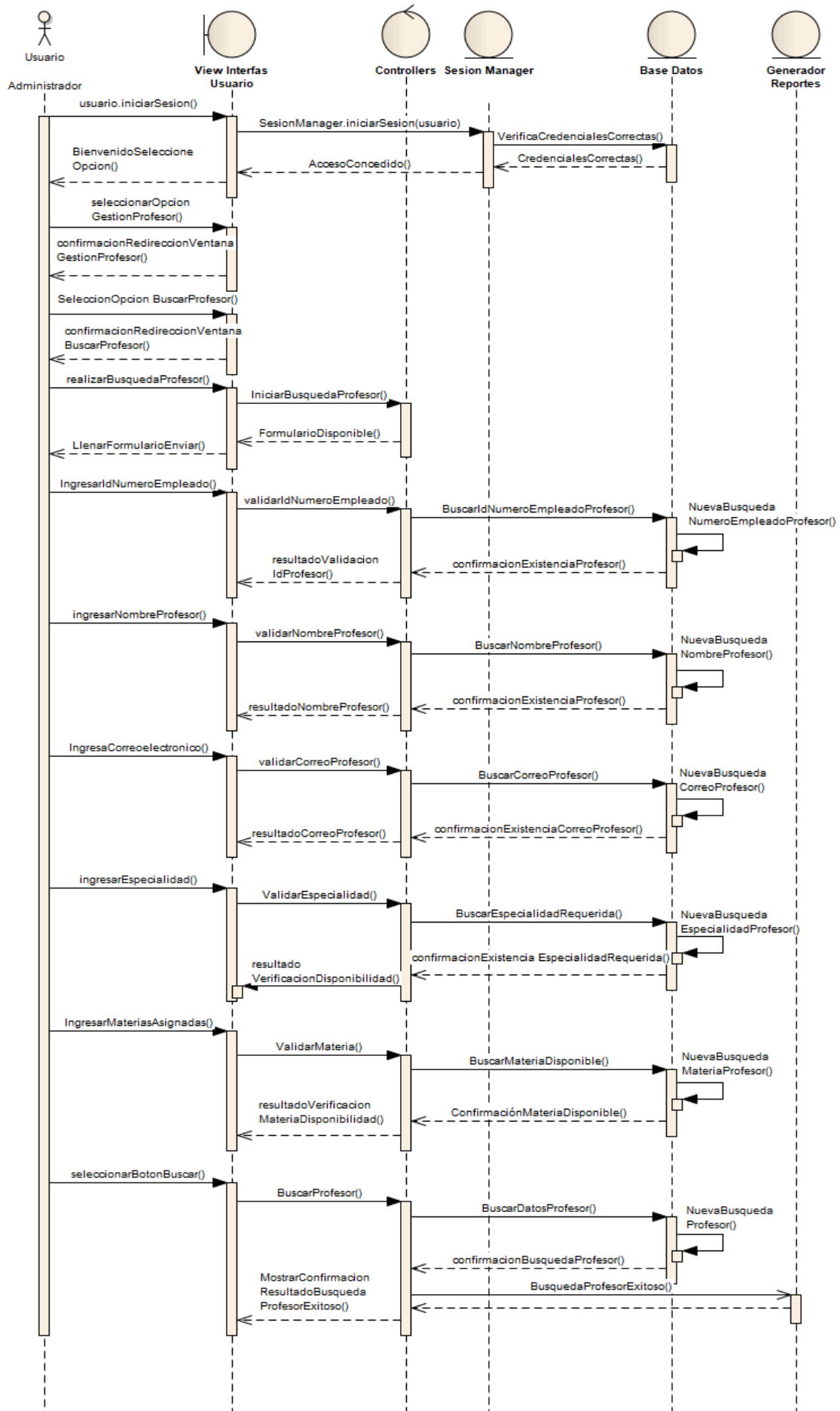
Versión	Fecha	Realizado por	Tiempo requerido
1.0	02/05/25	Cuapantecatl Ruiz Raul Irving	3:00 hrs

El diagrama de secuencia ilustra el flujo de interacciones entre los diferentes componentes del sistema durante el proceso de agregar un nuevo profesor. El proceso se inicia cuando el Administrador, a través de la interfaz de usuario, selecciona la opción para iniciar la gestión de profesores. La interfaz muestra una pantalla de bienvenida con varias opciones, incluyendo la de agregar un nuevo profesor, la cual el Administrador selecciona. Al seleccionar la opción de agregar profesor, la interfaz solicita al Administrador que llene un formulario con los datos del nuevo profesor, como el ID de empleado, nombre, correo electrónico, especialidad y materias asignadas. A medida que el Administrador ingresa cada dato, la interfaz envía mensajes al componente "Controllers" para validar la información ingresada. El "Controllers" a su vez interactúa con el "SesionManager" y la "Base Datos" para realizar las validaciones necesarias, como verificar si ya existe un profesor con el mismo ID, nombre o correo electrónico, y para confirmar la existencia y disponibilidad de la especialidad y las materias asignadas.

Una vez que todos los datos han sido ingresados y validados con éxito, el Administrador selecciona el botón "Registrar". La interfaz envía un mensaje "RegistrarProfesor()" al "Controllers", el cual inicia el proceso de guardar los datos del nuevo profesor en la "Base Datos" mediante el mensaje "GuardarDatosProfesor()". La "Base Datos" confirma que el nuevo profesor ha sido agregado. Finalmente, el "Controllers" informa a la interfaz de usuario mediante el mensaje "MostrarConfirmacionRegistroProfesor()" que el registro del profesor ha sido exitoso, mostrando un mensaje de confirmación al Administrador. Adicionalmente, se registra un evento de "Nuevo UsuarioProfesor" en el componente "Registro Eventos". En resumen, el diagrama de secuencia detalla cómo la interfaz de usuario, el "Controllers", el "SesionManager", la "Base Datos" y el "Registro Eventos" colaboran para llevar a cabo la adición de un nuevo profesor al sistema, incluyendo la validación de los datos ingresados y la confirmación de la operación al Administrador.

9.2 DIAGRAMA DE SECUENCIA 2 DEL CASO DE USO 01 BUSCAR PROFESOR:

sd BuscarProfesor



El diagrama de secuencia ilustra cómo el Administrador interactúa con el sistema para buscar información de un profesor existente. El proceso comienza cuando el Administrador inicia sesión en el sistema a través de la interfaz de usuario. Después de una validación exitosa de sus credenciales por parte del "SesionManager", el acceso es concedido y la interfaz muestra las opciones disponibles.

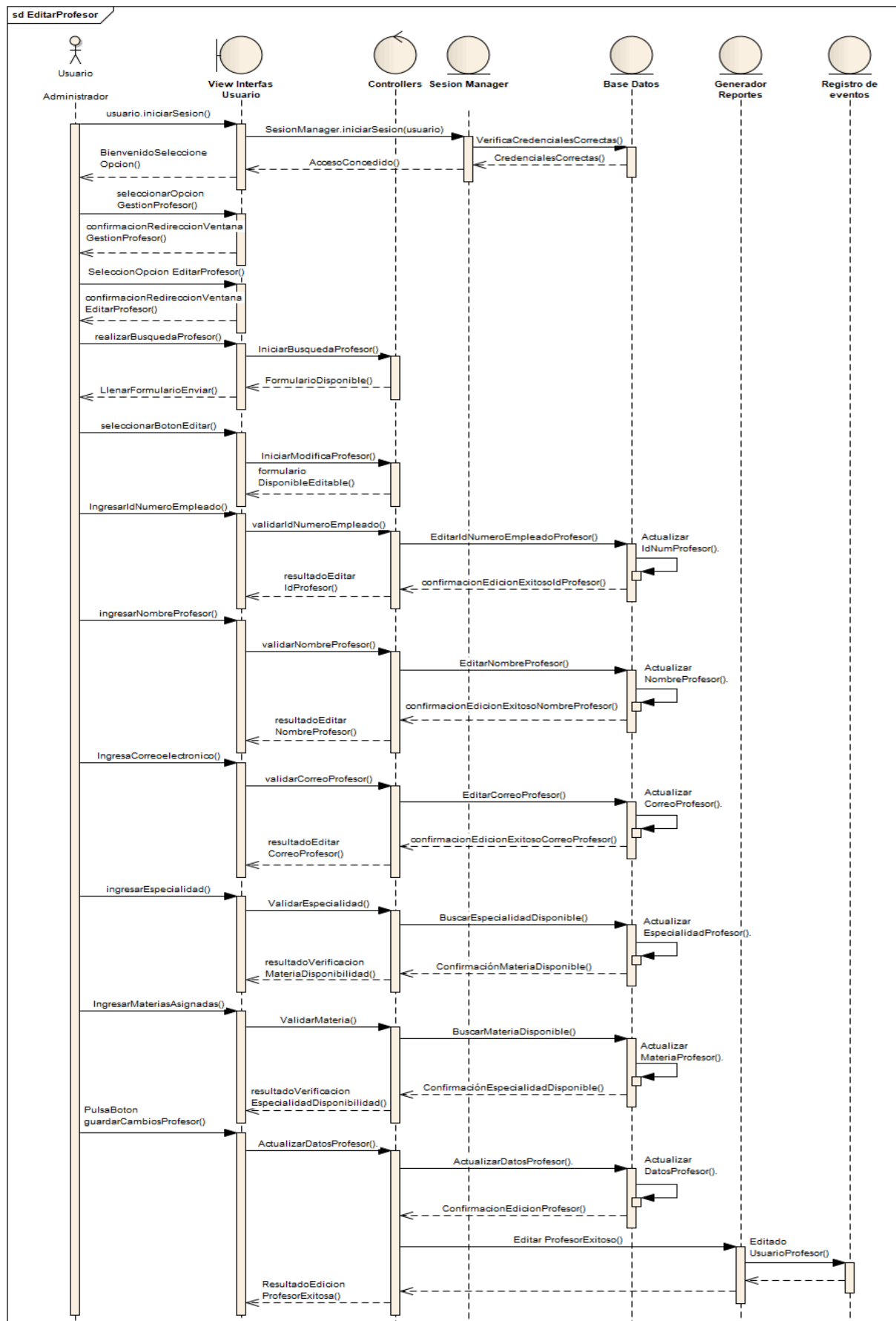
El Administrador navega y selecciona la opción para buscar un profesor. La interfaz presenta un formulario donde el Administrador puede ingresar criterios de búsqueda como el ID de empleado, nombre, correo electrónico, especialidad o materias asignadas. A medida que el Administrador introduce cada criterio y solicita la búsqueda, la interfaz envía estos datos al componente "Controllers".

El "Controllers" recibe los criterios de búsqueda y se comunica con la "Base Datos", enviando consultas específicas para buscar profesores que coincidan con la información proporcionada. Por ejemplo, si el Administrador ingresa un ID de empleado, el "Controllers" envía una consulta a la "Base Datos" para buscar un profesor con ese ID. La "Base Datos" realiza la búsqueda y devuelve los resultados al "Controllers".

Una vez que el "Controllers" recibe los resultados de la búsqueda desde la "Base Datos", formatea la información y la envía de vuelta a la interfaz de usuario. Finalmente, la interfaz muestra al Administrador la información del profesor o los profesores que coinciden con los criterios de búsqueda ingresados.

En resumen, el diagrama de secuencia detalla el flujo de información desde la interacción del Administrador con la interfaz, pasando por el "Controllers" que orquesta la búsqueda en la "Base Datos", hasta la presentación de los resultados de vuelta al Administrador.

9.3 DIAGRAMA DE SECUENCIA 3 DEL CASO DE USO 01 EDITAR PROFESOR:



Versión	Fecha	Realizado por	Tiempo requerido
1.0 DIAGRAMAS DE SECUENCIA (1,2,3)	08/05/25	Cuapantecatl Ruiz Raul Irving	8:00 hrs

El diagrama de secuencia ilustra el proceso mediante el cual un Administrador modifica la información de un profesor existente en el sistema. El proceso comienza con el Administrador iniciando sesión a través de la interfaz de usuario. Tras una autenticación exitosa gestionada por el "SesionManager", el sistema concede acceso y muestra las opciones disponibles.

El Administrador navega y selecciona la opción para editar la información de un profesor. La interfaz presenta una pantalla de búsqueda donde el Administrador puede encontrar al profesor que desea modificar. Una vez que el profesor es seleccionado para edición, la interfaz muestra un formulario pre cargado con la información actual del profesor.

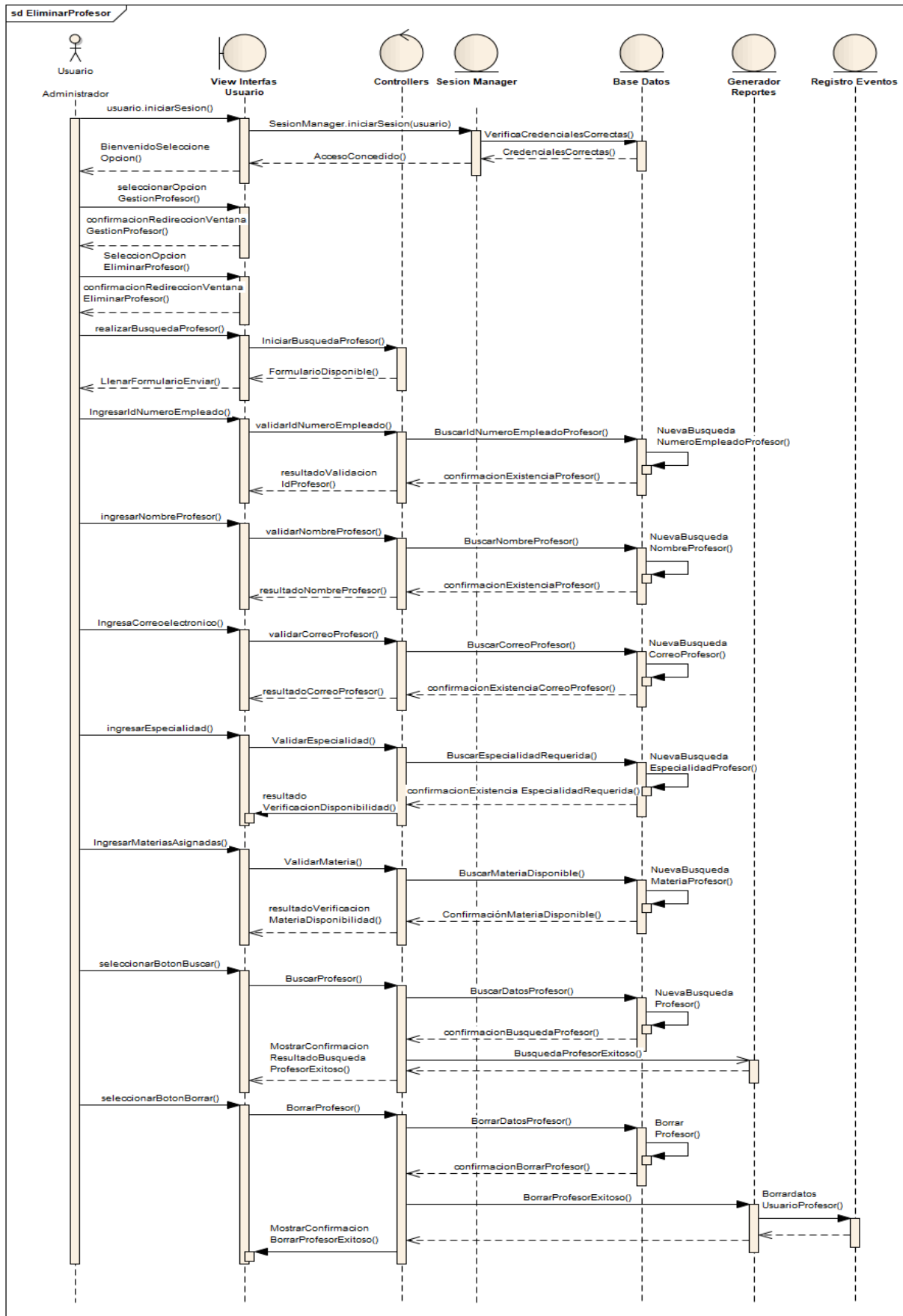
El Administrador puede entonces modificar los campos que necesite actualizar, como el número de empleado, nombre, correo electrónico, especialidad o las materias asignadas. A medida que el Administrador realiza cambios en cada campo y pulsa el botón para guardar los cambios, la interfaz envía los nuevos datos al componente "Controllers" para su validación. El "Controllers" interactúa con la "Base Datos" para verificar la validez de los nuevos datos, como la existencia de la especialidad y las materias.

Una vez que los datos modificados son validados exitosamente, el "Controllers" envía una solicitud a la "Base Datos" para actualizar la información del profesor mediante el mensaje "ActualizarDatosProfesor()". La "Base Datos" realizó la actualización y confirmó la edición exitosa.

Finalmente, el "Controllers" informa a la interfaz de usuario mediante el mensaje "ResultadoEdicionProfesor()" que la información del profesor ha sido actualizada correctamente, mostrando un mensaje de confirmación al Administrador. Adicionalmente, se registra un evento de "UsuarioProfesor Editado" en el componente "Registro de eventos".

En resumen, el diagrama de secuencia detalla cómo la interfaz de usuario, el "Controllers", el "SessionManager" y la "Base Datos" colaboran para permitir al Administrador editar la información de un profesor, incluyendo la búsqueda del profesor, la validación de los cambios y la actualización de la base de datos, culminando con una confirmación al usuario.

9.4 DIAGRAMA DE SECUENCIA 4 DEL CASO DE USO 01 BORRAR PROFESOR:



Versión	Fecha	Realizado por	Tiempo requerido
1.0 DIAGRAMAS DE SECUENCIA (4)	16/05/25	Cuapantecatl Ruiz Raul Irving	2:00 hrs

El diagrama de secuencia ilustra el proceso mediante el cual un Administrador elimina la información de un profesor existente del sistema. El proceso comienza con el Administrador iniciando sesión en el sistema a través de la interfaz de usuario. Tras una autenticación exitosa gestionada por el "SesionManager", el sistema concede acceso y muestra las opciones disponibles.

El Administrador navega y selecciona la opción para eliminar un profesor. La interfaz presenta una pantalla de búsqueda donde el Administrador puede encontrar al profesor que desea eliminar, utilizando criterios como el número de empleado, nombre, correo electrónico, especialidad o materias asignadas. Una vez que el Administrador introduce los criterios y pulsa el botón de buscar, la interfaz envía la solicitud al "Controllers".

El "Controllers" se comunica con la "Base Datos" para buscar al profesor que coincida con los criterios proporcionados. Una vez que el profesor a eliminar es encontrado y mostrado en la interfaz, el Administrador selecciona la opción de borrar o eliminar a ese profesor.

Al seleccionar la opción de borrar, la interfaz envía una solicitud "BorrarProfesor()" al "Controllers". El "Controllers" a su vez interactúa con la "Base Datos" enviando el mensaje "BorrarDatosProfesor()", indicando que se elimine la información del profesor seleccionado. La "Base Datos" realiza la eliminación y confirma la operación exitosa.

Finalmente, el "Controllers" informa a la interfaz de usuario mediante el mensaje "MostrarConfirmacionBorrarProfesor()" que el profesor ha sido eliminado correctamente, mostrando un mensaje de confirmación al Administrador. Adicionalmente, se registra un evento de "Borrado UsuarioProfesor" en el componente "Registro de eventos".

En resumen, el diagrama de secuencia detalla cómo la interfaz de usuario, el "Controllers" y la "Base Datos" colaboran para permitir al Administrador eliminar la

información de un profesor del sistema, incluyendo la búsqueda del profesor a eliminar y la confirmación de la operación.

10. Dinámica de estados (5.11).

Diagrama de máquina de estados UML, diagrama de estados (de Harel), tabla de transición de estados (matriz), autómatas, red de Petri. Transformación dinámica de estados. La Dinámica de Estados detalla cómo los elementos clave del sistema, como los profesores, las materias o los horarios, evolucionan a través de diferentes fases a lo largo del tiempo.

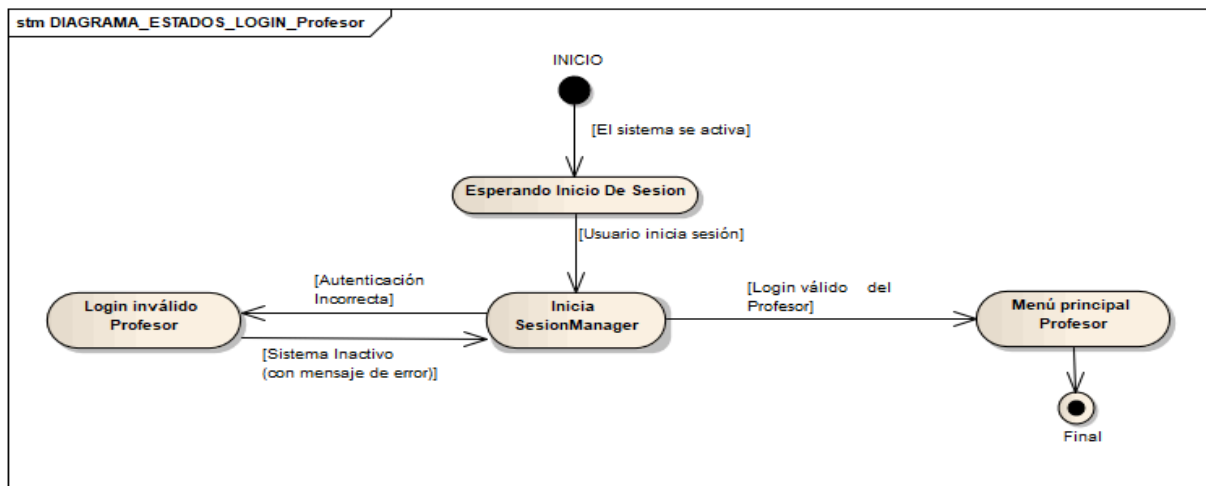
Para modelar este comportamiento, se utiliza el Diagrama de Máquina de Estados UML, que visualiza los distintos estados por los que un objeto puede pasar y las transiciones entre ellos, impulsadas por eventos específicos. Una extensión más potente para sistemas complejos es el Diagrama de Estados de Harel, que introduce conceptos adicionales para una representación más rica.

La Tabla de Transición de Estados (matriz) ofrece una perspectiva tabular, mostrando de forma concisa los posibles estados y cómo un evento particular causa el cambio a un nuevo estado.

Los Autómatas, como modelos computacionales abstractos, pueden ayudar a describir el comportamiento de los componentes del sistema. Por otro lado, la Red de Petri es útil para modelar el flujo de actividades, especialmente si existen procesos que ocurren de manera simultánea dentro del sistema de asignación.

Finalmente, la Transformación Dinámica de Estados abarca todo el proceso por el cual los objetos del sistema cambian de un estado a otro, identificando los eventos desencadenantes y las acciones resultantes de estas transiciones. En conjunto, estas herramientas permiten documentar de manera precisa y completa el comportamiento dinámico del sistema de asignación de horarios.

10.1 DIAGRAMA (1) ESTADOS LOGIN PROFESOR



El diagrama ilustra cómo el sistema maneja el proceso de inicio de sesión de un profesor. El diagrama de estados describe el ciclo de vida del proceso de inicio de sesión para un profesor dentro del sistema. El proceso comienza cuando el sistema se activa, lo que lleva al sistema al estado "Esperando Inicio De Sesión". En este punto, el sistema está inactivo, esperando que el profesor comience el proceso de inicio de sesión.

Cuando el profesor intenta iniciar sesión, el sistema pasa al estado "Inicia SesionManager". Aquí, el sistema valida las credenciales proporcionadas por el profesor. Si la autenticación falla, el sistema transita al estado "Login inválido Profesor", donde se muestra un mensaje de error al profesor, indicando que las credenciales no son válidas.

Si la autenticación es exitosa, el sistema avanza al estado "Menú principal Profesor". Esto significa que el profesor ha iniciado sesión correctamente y ahora tiene acceso a las funcionalidades del sistema a las que tiene permiso. Finalmente, el proceso de inicio de sesión termina en el estado "Final".

10.2 DIAGRAMA (2) ESTADOS LOGIN ADMINISTRADOR

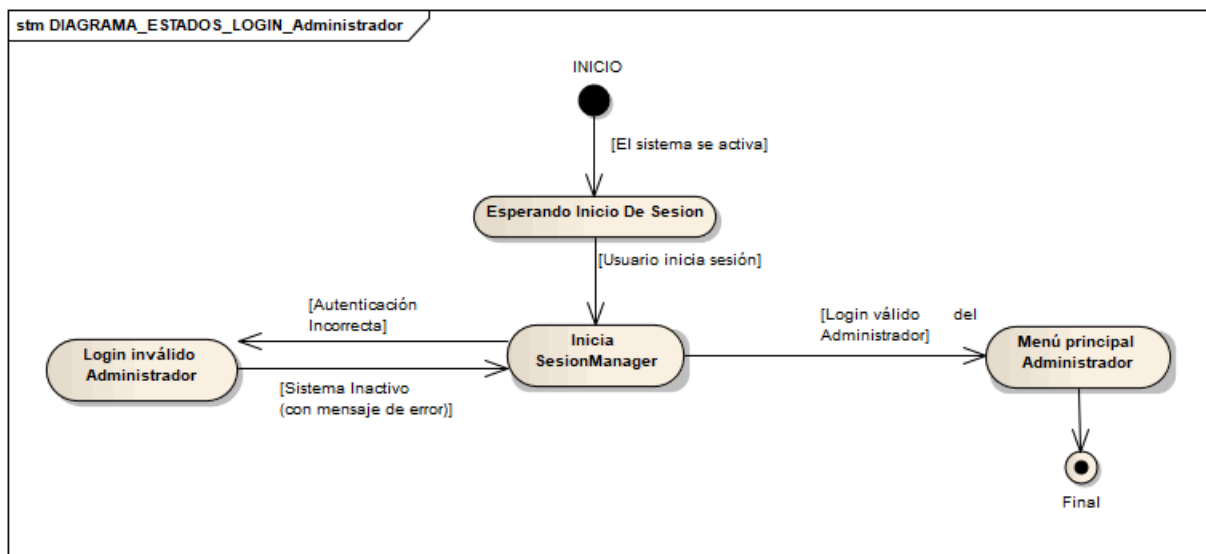


Diagrama de Estados de Inicio de Sesión del Administrador

El diagrama de estados para el Administrador es conceptualmente muy similar al del Profesor, pero con algunas diferencias clave en los estados o transiciones que podrían existir en un sistema más completo.

El proceso comienza cuando el sistema se activa, pasando al estado "Esperando Inicio de Sesión". El Administrador ingresa sus credenciales, y el sistema pasa al estado "Inicia SesionManager" para la validación. Si la autenticación falla, el sistema va a "Login inválido Administrador" y muestra un mensaje de error. Si tiene éxito, el sistema transita a "Menú principal Administrador", donde el Administrador tiene acceso a las funciones administrativas. Finalmente, el proceso termina en el estado "Final".

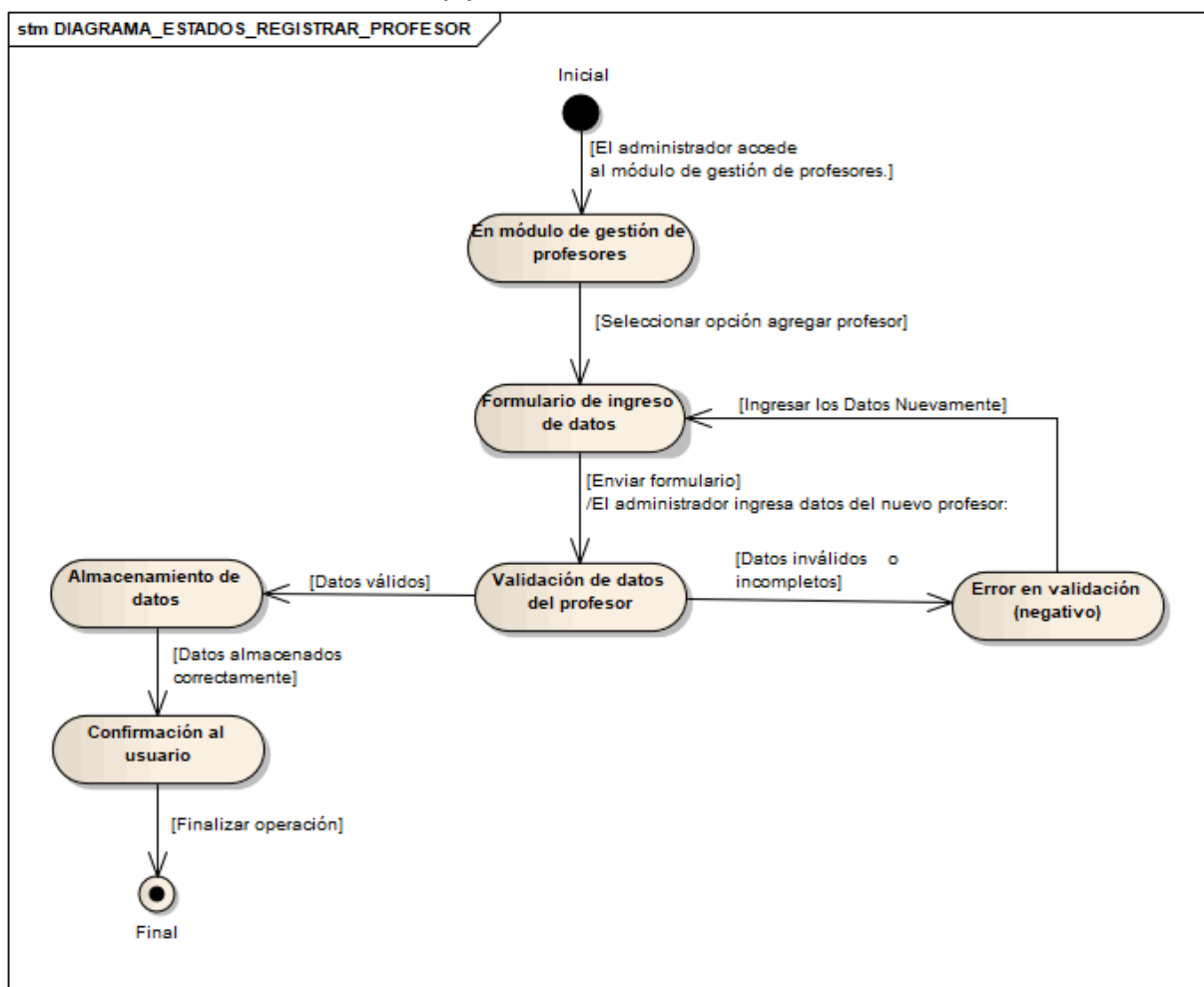
Importancia de Diferenciar los Logins de Profesor y Administrador

Es crucial separar los procesos de inicio de sesión para Profesor y Administrador por las siguientes razones:

- **Control de Acceso:** Los Administradores tienen permisos mucho más amplios que los Profesores. Separar los logins permite aplicar estos diferentes niveles de acceso.

- Seguridad: Forzar a los usuarios a iniciar sesión a través de rutas separadas reduce el riesgo de que un usuario con privilegios de Profesor acceda a funciones administrativas.
- Experiencia de Usuario: Los menús y las opciones disponibles después del inicio de sesión son diferentes para cada rol. La separación garantiza que cada usuario vea solo las funciones relevantes para él.
- Auditoría: Mantener registros de inicio de sesión separados para cada rol facilita el seguimiento de las acciones realizadas por cada tipo de usuario.

10.3 DIAGRAMA (3) ESTADOS REGISTRAR PROFESOR



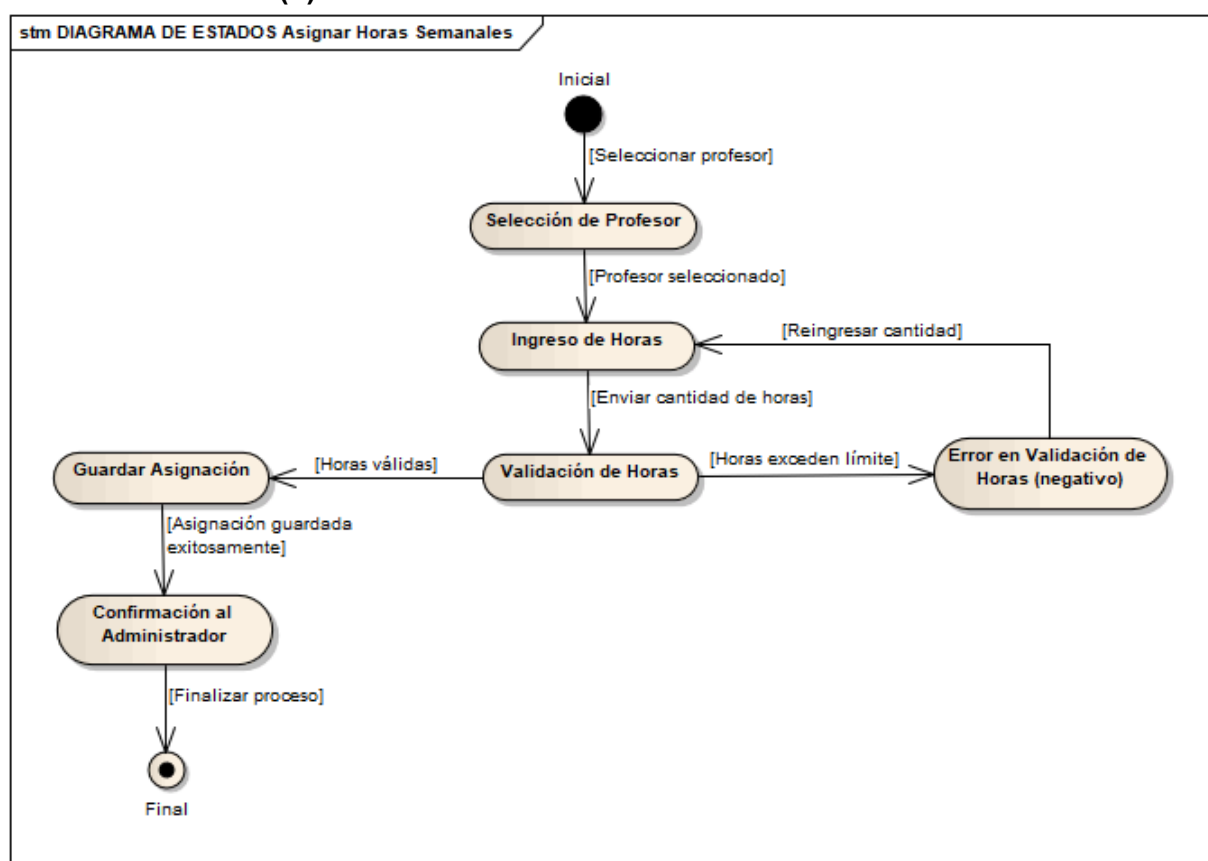
El diagrama de estados (3) describe el proceso de registro de un nuevo profesor en el sistema. El proceso comienza en el estado "Inicial". El Administrador accede al módulo de gestión de profesores, lo que lleva al sistema al estado "En módulo de gestión de profesores". Luego, el Administrador selecciona la opción para agregar

un profesor, y el sistema pasa al estado "Formulario de ingreso de datos", donde el Administrador introduce la información del nuevo profesor.

Una vez que el Administrador envía el formulario, el sistema entra en el estado "Validación de datos del profesor". Aquí, el sistema verifica que los datos ingresados sean válidos y completos. Si los datos son inválidos o incompletos, el sistema pasa al estado "Error en validación (negativo)", y el Administrador debe corregir los datos en el formulario.

Si los datos son válidos, el sistema pasa al estado "Almacenamiento de datos", donde la información del profesor se guarda en la base de datos. Si los datos se almacenan correctamente, el sistema pasa al estado "Confirmación al usuario", donde se muestra un mensaje de éxito al Administrador. Finalmente, la operación de registro termina en el estado "Final".

10.4 DIAGRAMA (4) ESTADOS ASIGNAR HORAS SEMANALES A PROFESOR



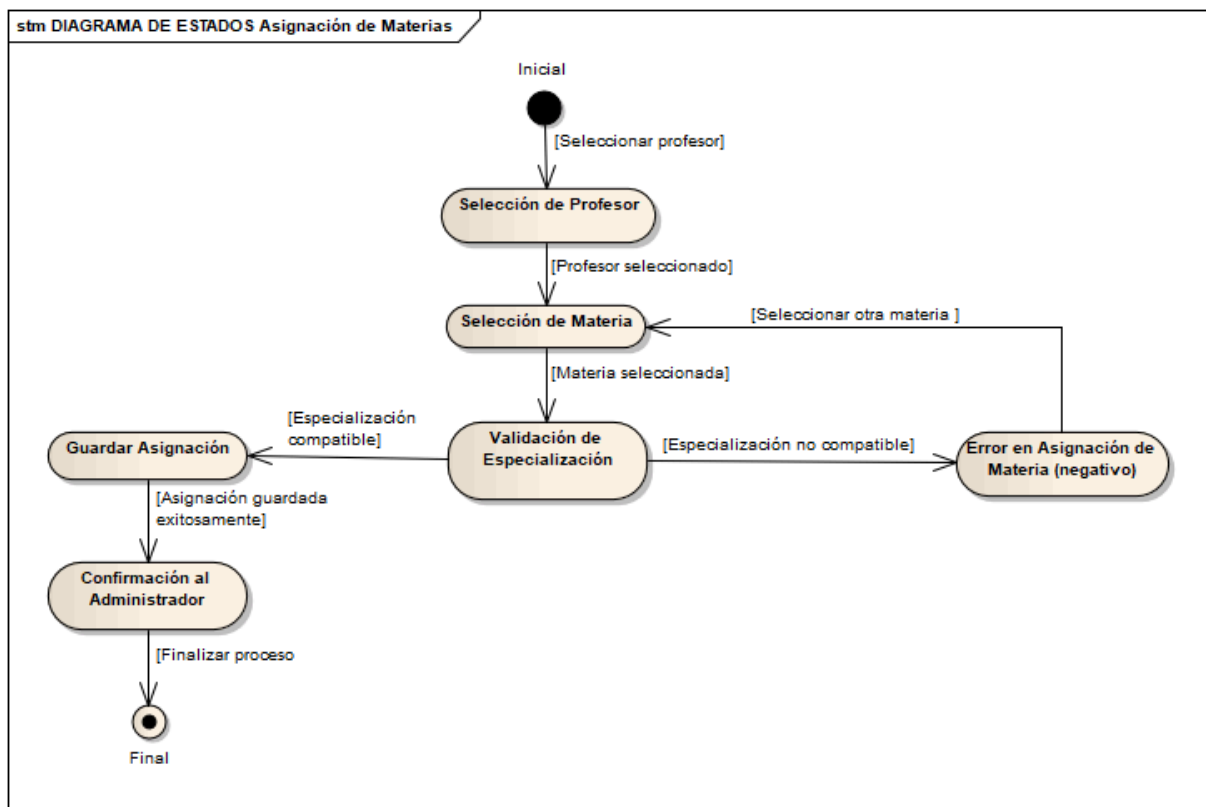
El diagrama de estados describe el proceso de asignar la cantidad de horas semanales que un profesor debe impartir. El proceso comienza en el estado "Inicial". El Administrador comienza seleccionando un profesor, lo que lleva al sistema al

estado "Selección de Profesor". Una vez que se ha seleccionado un profesor, el sistema pasa al estado "Ingreso de Horas", donde el Administrador introduce la cantidad de horas que se le asignan al profesor.

Después de que se introduce la cantidad de horas, el sistema entra en el estado "Validación de Horas". Aquí, el sistema verifica que la cantidad de horas introducidas sea válida y no exceda el límite permitido. Si la cantidad de horas excede el límite, el sistema pasa al estado "Error en Validación de Horas (negativo)", y el Administrador debe re ingresar una cantidad válida.

Si la cantidad de horas es válida, el sistema pasa al estado "Guardar Asignación", donde la asignación de horas se guarda en la base de datos. Si la asignación se guarda exitosamente, el sistema pasa al estado "Confirmación al Administrador", donde se muestra un mensaje de éxito. Finalmente, el proceso de asignación de horas termina en el estado "Final".

10.5 DIAGRAMA (5) ESTADOS ASIGNAR MATERIAS A PROFESOR



El diagrama de estados describe el proceso de asignar una materia a un profesor. El proceso comienza en el estado "Inicial". El Administrador comienza seleccionando

un profesor, lo que lleva al sistema al estado "Selección de Profesor". Una vez que se ha seleccionado un profesor, el sistema pasa al estado "Selección de Materia", donde el Administrador elige la materia que se le asignará al profesor. El Administrador puede seleccionar otra materia si lo desea, permaneciendo en este estado.

Después de que se selecciona una materia, el sistema entra en el estado "Validación de Especialización". Aquí, el sistema verifica que la especialización del profesor sea compatible con la materia seleccionada. Si la especialización no es compatible, el sistema pasa al estado "Error en Asignación de Materia (negativo)", y el Administrador debe seleccionar una materia diferente.

Si la especialización es compatible, el sistema pasa al estado "Guardar Asignación", donde la asignación de la materia al profesor se guarda en la base de datos. Si la asignación se guarda exitosamente, el sistema pasa al estado "Confirmación al Administrador", donde se muestra un mensaje de éxito. Finalmente, el proceso de asignación de la materia termina en el estado "Final".

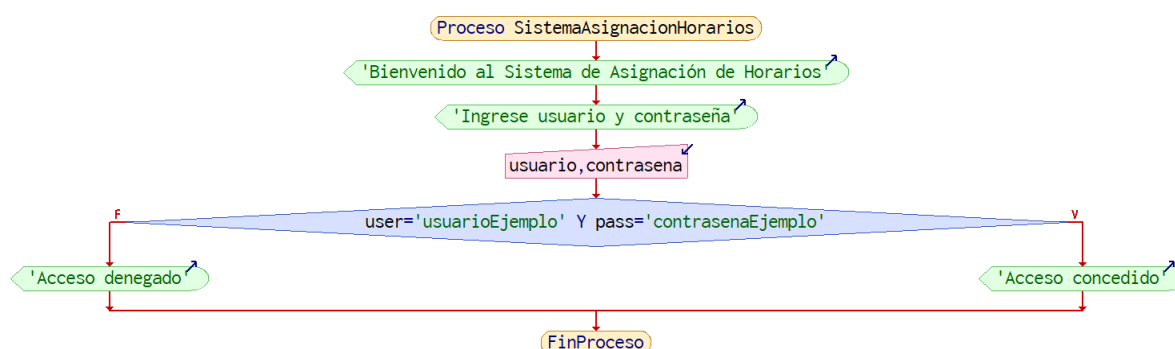
Versión	Fecha	Realizado por	Tiempo requerido
1.0 DIAGRAMA (5.11.1, 5.11.2, 5.11.3, 5.11.4, 5.11.5)	02/05/25	Cuapantecatl Ruiz Raul Irving	8:00 hrs

11. Lógica procedimental algorítmica (5.12)

Esta sección del Documento de Diseño de Software se centra en detallar la lógica y los procedimientos paso a paso que el sistema implementará para llevar a cabo sus diversas funcionalidades. Va más allá de las descripciones generales y profundiza en los algoritmos específicos que se utilizarán. Técnicas como tablas de decisión, diagramas de Warnier, JSP (Programación Estructurada de Jackson) y PDL (Lenguaje de Diseño de Programas) son herramientas que se utilizan para representar esta lógica procedimental de forma clara y estructurada, facilitando la comprensión, la implementación y el mantenimiento del comportamiento del sistema.

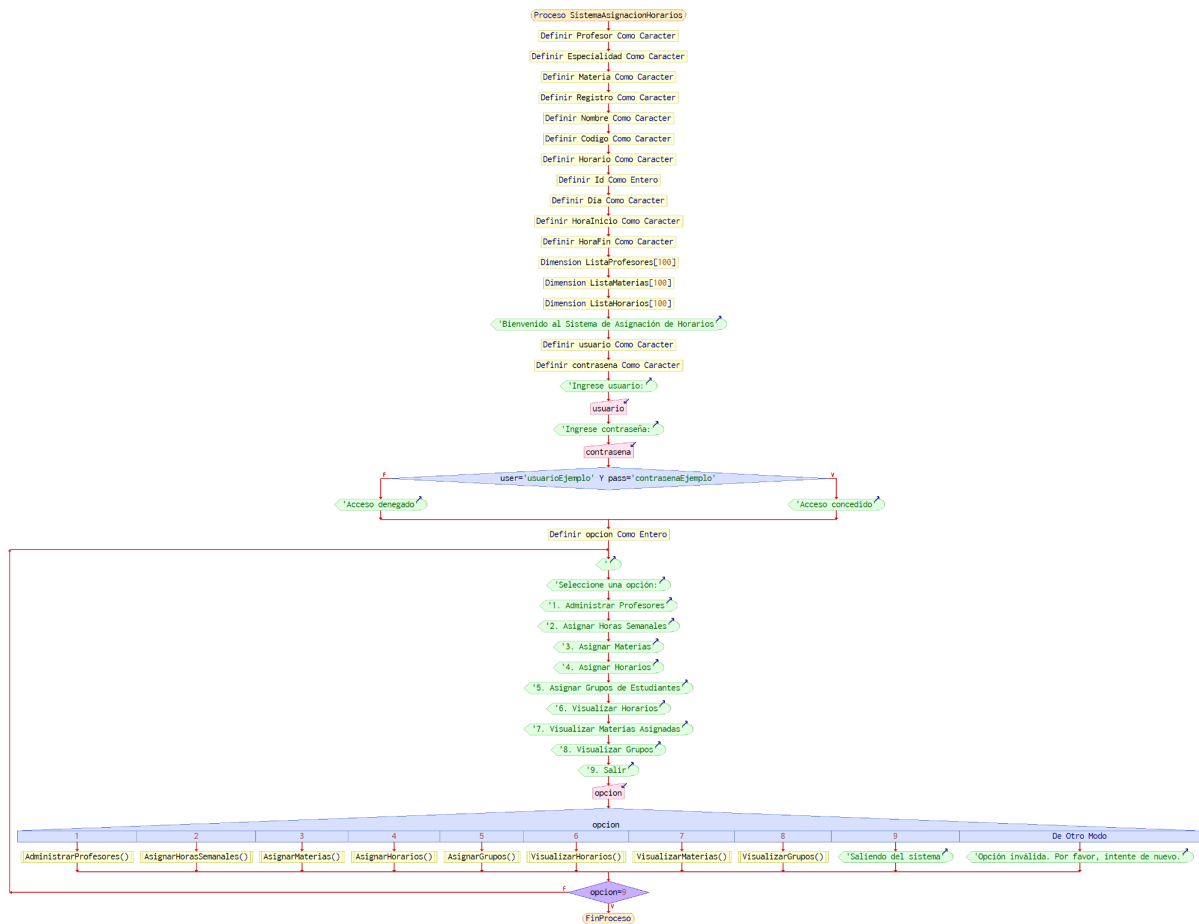
11.1 Algoritmo 1: Proceso SistemaAsignacionHorarios() (Iniciar sesión)

Este algoritmo describe el proceso de inicio de sesión inicial para todo el sistema de programación. Muestra un mensaje de bienvenida y solicita al usuario que ingrese su nombre de usuario y contraseña, los almacena en las variables `` usuario y contraseña``. Se toma una decisión: si el nombre de usuario ingresado es ``usuarioEjemplo`` y la contraseña es ``contrasenaEjemplo``, se concede el acceso; de lo contrario, se deniega. El proceso finaliza.



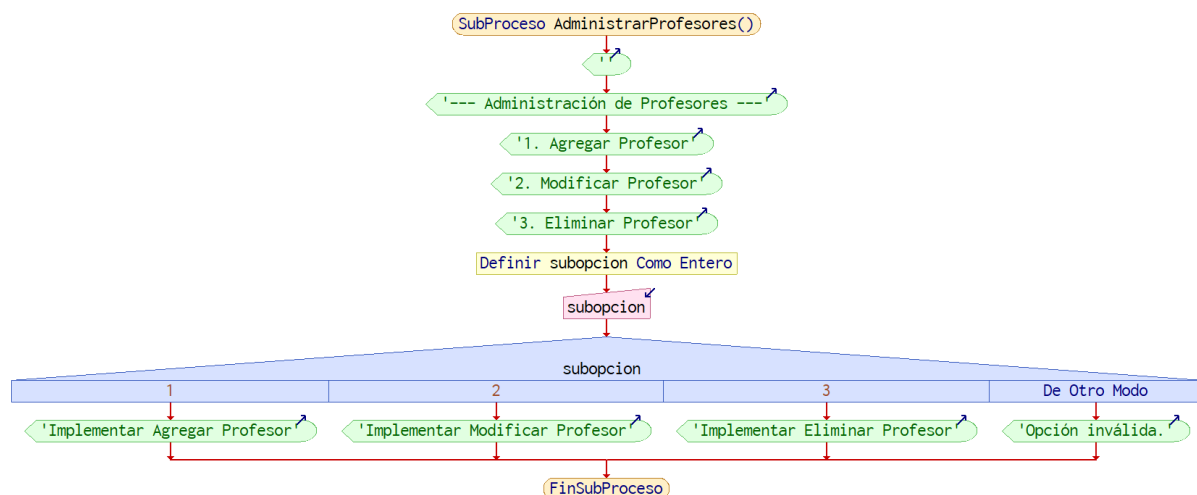
11.2 Algoritmo 2: Proceso SistemaAsignacionHorarios() (Menú Principal Para Administrador)

Este algoritmo amplía el proceso de inicio de sesión y presenta el menú principal del sistema. Tras la bienvenida inicial y el inicio de sesión (como se describe en el Algoritmo 1), opción variable. Se define una variable entera para almacenar la selección del menú del usuario. El sistema muestra una lista numerada de opciones disponibles, que incluyen la gestión de profesores, la asignación de horas semanales, la asignación de asignaturas, la asignación de horarios, la asignación de grupos de estudiantes, la visualización de horarios, la visualización de asignaturas asignadas, la visualización de grupos asignados y la salida. Se solicita al usuario que introduzca su selección. Una estructura de selección multidireccional (implícita en la tabla debajo del menú) dirige el flujo del programa al subproceso correspondiente según el valor de opción. También existe un caso predeterminado de "opción no válida".



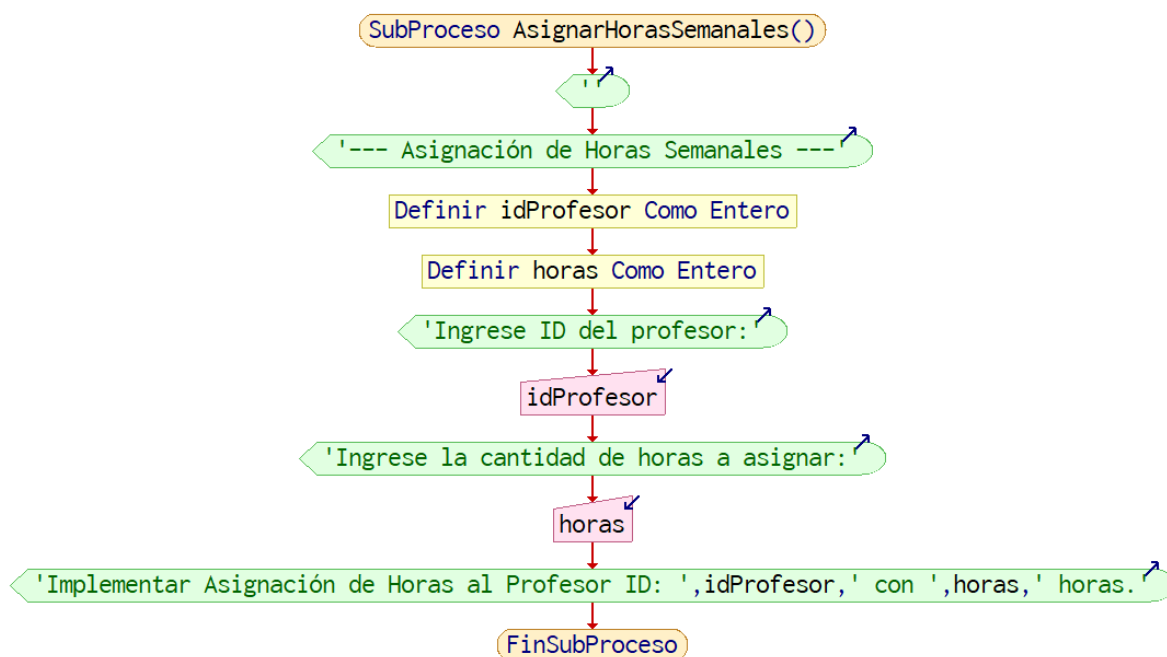
11.3 Algoritmo 3: SubProceso AdministrarProfesores()

Este algoritmo detalla las opciones disponibles para la gestión de profesores. Muestra un menú con opciones para añadir, modificar y eliminar profesores. subopción. Se define una variable entera para almacenar la elección del usuario. Una estructura de selección multidireccional dirige el flujo del programa al subproceso "Implementar Agregar Profesor", "Implementar Modificar Profesor" o "Implementar Eliminar Profesor" según el valor de subopción. También se puede usar una opción inválida.



11.4 Algoritmo 4: SubProceso AsignarHorasSemanales()

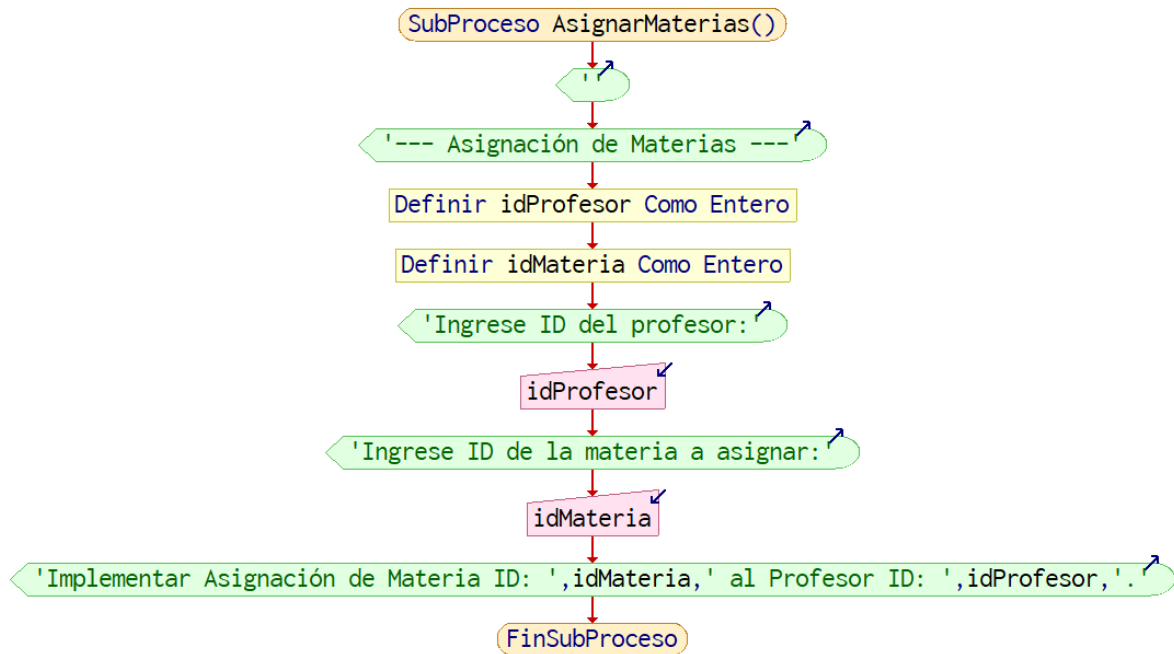
Este algoritmo describe el proceso de asignación de horas semanales a un profesor. Comienza definiendo variables enteras para el ID del profesor (idProfesor) y el número de horas a asignar (horas). A continuación, se solicita al usuario que introduzca el ID del profesor y el número de horas. Finalmente, indica que se realizará la asignación de las horas introducidas al profesor especificado.



11.5 Algoritmo 5: SubProceso AsignarMaterias()

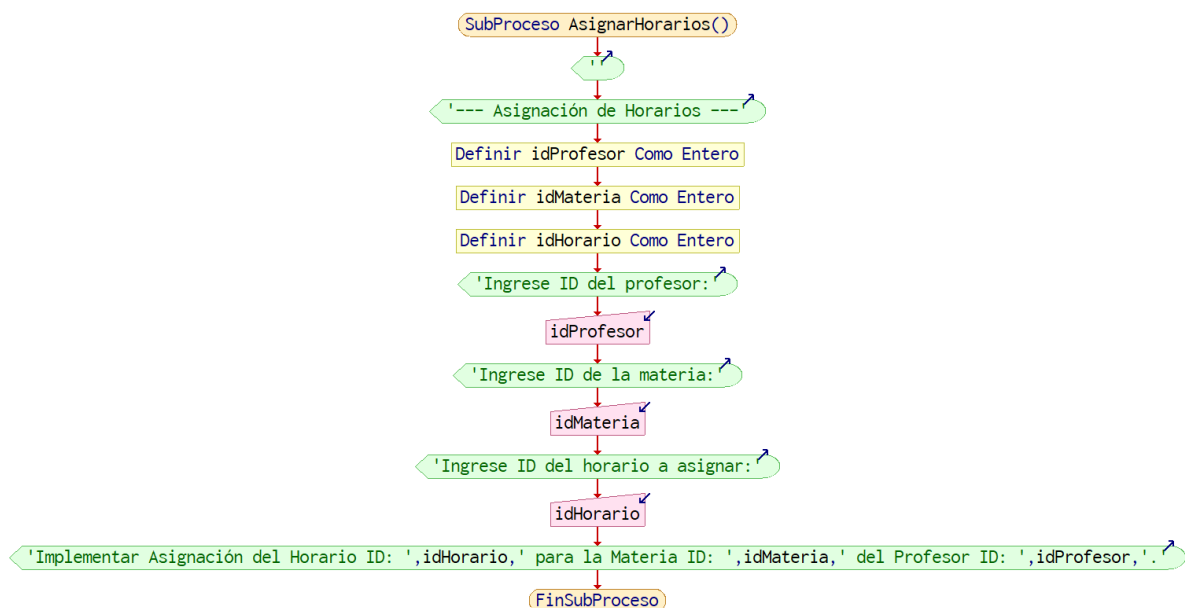
Este algoritmo describe los pasos para asignar una asignatura a un profesor. Comienza definiendo variables enteras para el ID del profesor (idProfesor) y el ID de la asignatura (idMateria). A continuación, se solicita al usuario que introduzca el

ID del profesor y el ID de la asignatura que se asignará. El paso final indica que se realizará la asignación de la asignatura especificada al profesor dado.



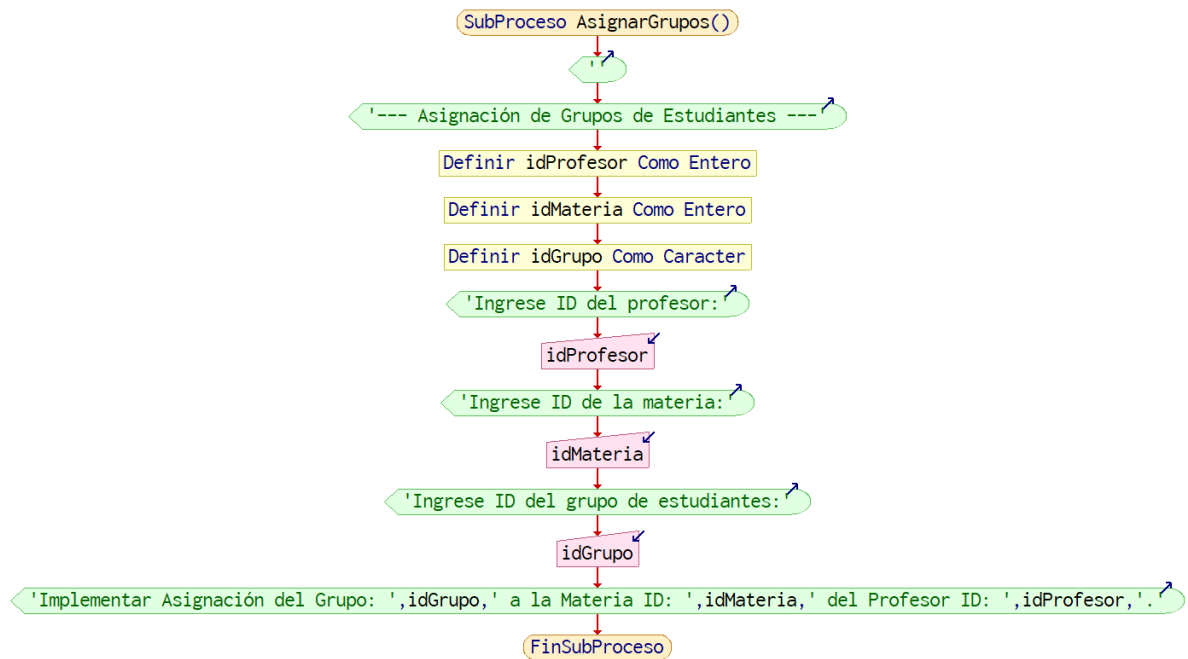
11.6 Algoritmo 6: SubProceso AsignarHorarios()

Este algoritmo describe el proceso para asignar un horario a un profesor para una materia específica. Primero, se definen las variables para almacenar el ID del profesor, el ID de la materia y el ID del horario a asignar. Luego, se solicita al usuario que ingrese estos tres ID. Finalmente, se indica que se debe implementar la lógica para realizar la asignación del horario a la materia y al profesor utilizando las identificaciones proporcionadas.



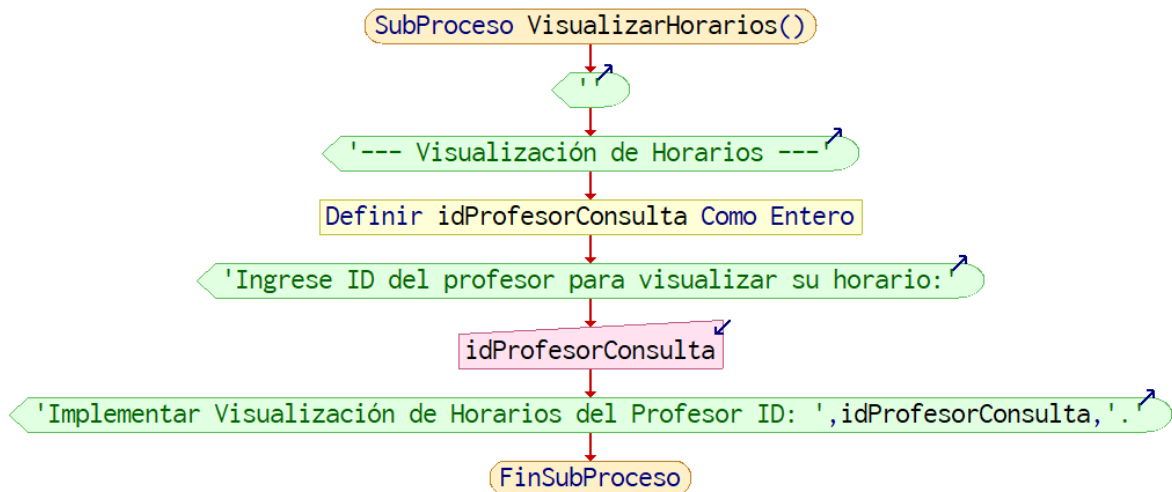
11.7 Algoritmo 7: SubProceso AsignarGrupos()

Este algoritmo describe el proceso para asignar un grupo de estudiantes a un profesor para una materia específica. Primero, se definen las variables para almacenar el ID del profesor, el ID de la materia y el ID del grupo. Luego, se solicita al usuario (presumiblemente el administrador) que ingrese estos tres ID. Finalmente, se indica que se debe implementar la lógica para realizar la asignación del grupo a la materia y al profesor utilizando las identificaciones proporcionadas.



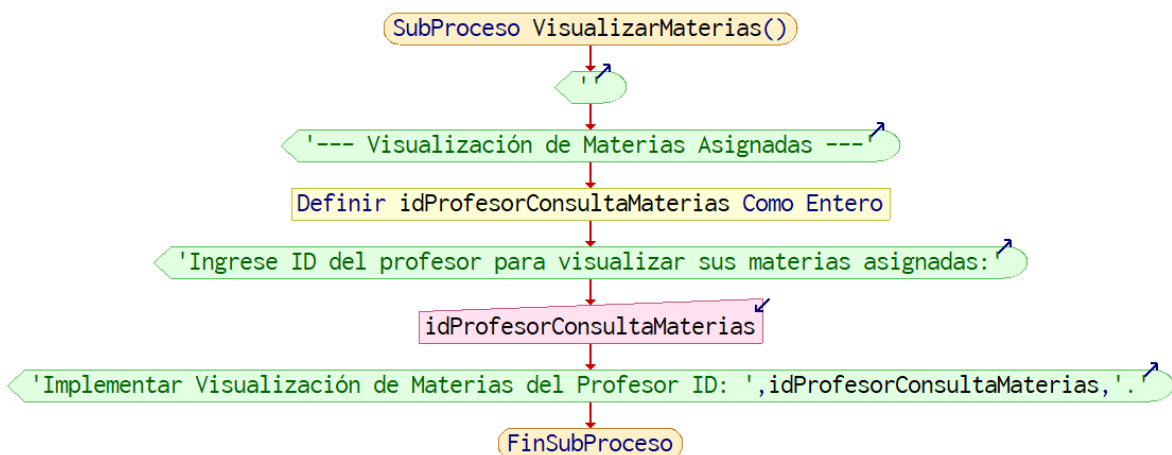
11.8 Algoritmo 8: SubProceso VisualizarHorarios()

Este algoritmo detalla cómo un Administrador puede visualizar el horario de un profesor. Se define una variable para almacenar el ID del profesor que se desea consultar. Se solicita al usuario que ingrese el ID del profesor. Finalmente, se señala que se debe implementar la lógica para recuperar y mostrar el horario correspondiente al ID del profesor ingresado.



11.9 Algoritmo 9: SubProceso VisualizarMaterias()

Este algoritmo explica cómo un administrador puede ver las materias que tiene un profesor asignadas. Se define una variable para almacenar el ID del profesor cuya lista de materias se quiere visualizar. Se solicita al usuario que ingrese el ID del profesor. Finalmente, se indica que se debe implementar la lógica para obtener y mostrar las materias asignadas al ID del profesor ingresado.



11.10 Algoritmo 10: SubProceso VisualizarGrupos()

Este algoritmo describe cómo un administrador puede ver los grupos de estudiantes que tiene un profesor designados. Se define una variable para almacenar el ID del profesor cuyos grupos se quieren visualizar. Se solicita al usuario que ingrese el ID del profesor. Finalmente, se señala que se debe implementar la lógica para recuperar y mostrar los grupos asignados al ID del profesor ingresado.

11.11 Algoritmo 11: Gale_Shapley_AsignacionMaterias()

el algoritmo implementado en el pseudocódigo sigue la lógica del algoritmo de Gale-Shapley:

- Los profesores proponen las materias en orden de su preferencia.
- Las materias aceptan la primera propuesta si están libres.
- Si una materia recibe una propuesta de un profesor que prefiere más que a su profesor actual, lo reemplaza.
- El proceso continúa hasta que se alcanza una asignación estable donde ningún profesor y ninguna materia preferirían estar emparejados entre sí en lugar de su asignación actual.

Las estructuras de datos preferenciaProfesores y preferenciaMaterias son cruciales para definir las preferencias de cada entidad y guiar el proceso de emparejamiento. El uso de proximoIntentoProfesores asegura que cada profesor avance en su lista de preferencias en cada propuesta hasta encontrar una pareja o agotar sus opciones.



Versión	Fecha	Realizado por	Tiempo requerido
0.1 Anexo pseudocódigo de los algoritmos (1,2,3,4,5,6,7,8,9,10, 11)	16/005/25	Cuapantecatl Ruiz Raul Irving	6:00 hrs

12.Conclusión

El desarrollo del "Sistema de Asignación de Horarios de Profesores" evidencia una sólida planificación y un enfoque profesional en su diseño, como lo demuestra el documento SDD. La organización meticulosa del contenido, la claridad en la identificación de actores y casos de uso, y el uso extensivo de diagramas UML reflejan una comprensión integral del sistema y sus requerimientos. Además, la incorporación de patrones de diseño reconocidos y la distinción entre las vistas lógica y física de la arquitectura aseguran una base técnica robusta, orientada a la escalabilidad y el mantenimiento. En conjunto, estos elementos indican que el proyecto está bien encaminado hacia el desarrollo de una solución eficiente y confiable, con un compromiso evidente hacia la calidad, la documentación y las buenas prácticas de ingeniería de software.

13. ANEXOS

Resumen:

El objetivo es mejorar la gestión académica al automatizar la asignación de profesores a materias y horarios, asegurando una distribución justa y optimizada de los recursos. El sistema incluirá una interfaz web para administradores y profesores, permitiendo la gestión eficiente de horarios y evitando conflictos.

El sistema de asignación de materias y horarios está diseñado para universidades y otras instituciones educativas. Incluirá:

- Gestión de profesores: Registro, asignación, modificación y eliminación de docentes.
- Asignación de materias: Distribución de asignaturas según disponibilidad y especialización.
- Horarios: Definición de clases sin solapamientos.
- Gestión de aulas: Maximización del uso de espacios.
- Reportes: Generación de informes sobre carga de trabajo y distribución de materias.
- Seguridad: Control de accesos y protección de datos.

Para la implementación del sistema se requiere:

Infraestructura

- Servidor web para alojar la aplicación.
- Base de datos SQL (MySQL, PostgreSQL) para almacenar información.
- Red estable para acceso a la plataforma.

Software y Tecnologías

- Backend: Python (Flask/Django) o Java (Spring Boot).
- Frontend: HTML, CSS, JavaScript (React, Vue o Angular).
- Base de datos: MySQL, PostgreSQL o equivalente.
- Seguridad: Implementación de autenticación (JWT, OAuth).

Requisitos y Restricciones

- Acceso controlado según rol de usuario (administrador, profesor).
- Evitar colisiones de horarios entre profesores y materias.
- Uso eficiente de aulas para evitar espacios desaprovechados.
- Escalabilidad para soportar futuros aumentos de carga académica.

Enfoque Algorítmico

Para optimizar la asignación de profesores, horarios y aulas, el sistema implementará algoritmos como Gale-Shapley para la asignación de profesores y programación lógica condicional con restricciones para horarios y aulas.

Beneficios del Sistema

- Automatización de procesos administrativos.
- Reducción de errores y conflictos de horarios.
- Optimización del uso de aulas y profesores.
- Escalabilidad para futuras expansiones.

Se ha determinado que para lograr una distribución eficiente de los recursos académicos, se empleará un enfoque basado en algoritmos de optimización.

Objetivos

- Garantizar una distribución equitativa de las materias entre los profesores.
- Evitar solapamientos de horarios en las asignaciones.
- Maximizar la utilización de las aulas.
- Implementar un sistema escalable y flexible para futuras modificaciones.

Algoritmos Seleccionados

Para cumplir con los objetivos planteados, el sistema utilizará una combinación de algoritmos:

1. Algoritmo de Emparejamiento Estable (Gale-Shapley)

- Se utilizará para asignar materias a profesores de manera justa y eficiente.
- Garantiza que cada profesor reciba asignaciones acorde a su disponibilidad y especialización.
- Considera preferencias y restricciones para una distribución óptima.

2. Programación con Restricciones (Constraint Programming)

- Define reglas estrictas para evitar colisiones de horarios.
- Asegura que una clase no pueda ser asignada a más de un aula o profesor a la vez.
- Permite ajustar restricciones de capacidad y disponibilidad de aulas.

Enfoque Algorítmico

El sistema implementará algoritmos para optimizar la asignación de profesores, horarios y aulas.

- Gale-Shapley:
 - Este algoritmo se utilizará para la asignación de profesores a materias, teniendo en cuenta las preferencias de los profesores y las necesidades del sistema.
 - Aplicación: se aplicará para la asignación de profesores a materias, tomando en cuenta las especializaciones de los profesores, y las necesidades de cada materia.
- Programación Lógica Condicional con Restricciones:

- Este enfoque se utilizará para la asignación de horarios y aulas, teniendo en cuenta restricciones como la disponibilidad de profesores, la capacidad de las aulas y las preferencias de los profesores.
- Aplicación: se aplicará para la asignación de los horarios, tomando en cuenta la disponibilidad de los profesores, que no se sobre pongan los horarios de un mismo profesor, y tomando en cuenta que la materia asignada corresponda a la especialización del profesor.

Aplicación del Enfoque Algorítmico

- Para la asignación de profesores a materias (CU03), el algoritmo Gale-Shapley se aplicará para encontrar una asignación estable que maximice la satisfacción de los profesores y las necesidades del sistema.
- Para la asignación de horarios (CU04), la programación lógica condicional con restricciones se aplicará para encontrar una solución que cumpla con todas las restricciones y optimice la utilización de los recursos.
- para la asignación de grupos de estudiantes (CU05) Se tomara en cuenta las restricciones de los horarios de los profesores, y la disponibilidad de los grupos, para que no se generen conflictos.

Este enfoque algorítmico permitirá al sistema realizar asignaciones eficientes y justas, optimizando la utilización de los recursos y maximizando la satisfacción de los usuarios.

Flujo de Asignación

1. Recolecta Información:
 - Lista de profesores y materias.
 - Disponibilidad horaria de los profesores.
 - Capacidad y disponibilidad de aulas.
2. Fase 1 - Asignación de Profesores a Materias (Gale-Shapley):
 - Se emparejan materias con profesores según su disponibilidad y especialidad.
 - Se priorizan profesores con mayor experiencia en ciertas asignaturas.
3. Fase 2 - Asignación de Horarios y Aulas (Constraint Programming):
 - Se evitan solapamientos de clases en horarios y aulas.

- Se buscan combinaciones óptimas para el uso de espacios disponibles.

Beneficios del Enfoque

- Evita conflictos de horarios y sobrecarga de trabajo en profesores.
- Maximiza la ocupación de aulas, evitando espacios vacíos.
- Permite escalabilidad para incluir nuevos profesores, materias y aulas.
- Reduce la carga administrativa en la asignación de horarios.